

Supplemental Notes for Bernoulli Factory Algorithms

Peter Occil

This version of the document is dated 2026-07-30.

Peter Occil

1 Introduction

The *Bernoulli factory problem* is: Suppose there is a coin that shows heads with an unknown probability, λ , and the goal is to use that coin (and possibly also a fair coin) to build a “new” coin that shows heads with a probability that depends on λ , call it $f(\lambda)$, where f is a known function. This article discusses additional topics on the Bernoulli factory problem and should be read in conjunction with the article “**Bernoulli Factory Algorithms**¹”.

2 Contents

- Introduction
- Contents
- About This Document
- Definitions
- General Factory Functions
 - Building the Lower and Upper Polynomials
 - Another General Algorithm
 - Request for Additional Methods
- Approximate Bernoulli Factories
- Achievable Simulation Rates
- Notes
- Appendix
 - Examples of Well-Behaved Functions
 - Results Used in Approximate Bernoulli Factories
 - How Many Coin Flips Are Needed to Simulate a Polynomial?
 - Proofs for Polynomial-Building Schemes
 - * A Conjecture on Polynomial Approximation
 - * Example of Polynomial-Building Scheme
 - Which functions don’t require outside randomness to simulate?
 - Multiple-Output Bernoulli Factory
 - Pushdown Automata and Algebraic Functions
 - * Finite-State and Pushdown Generators
- License

¹<https://peteroupc.github.io/bernoulli.html>

3 About This Document

This is an open-source document; for an updated version, see the source code² or its rendering on GitHub³. You can send comments on this document on the GitHub issues page⁴. See “Open Questions on the Bernoulli Factory Problem”⁵ for a list of things about this document that I seek answers to.

My audience for this article is **computer programmers with mathematics knowledge, but little or no familiarity with calculus**.

I encourage readers to implement any of the algorithms given in this page, and report their implementation experiences. In particular, **I seek comments on the following aspects**⁶:

- Are the algorithms in this article (in conjunction with “Bernoulli Factory Algorithms”) easy to implement? Is each algorithm written so that someone could write code for that algorithm after reading the article?
- Does this article have errors that should be corrected?
- Are there ways to make this article more useful to the target audience?

Comments on other aspects of this document are welcome.

4 Definitions

This section describes certain math terms used on this page for programmers to understand.

The *closed unit interval* (written as $[0, 1]$) means the set consisting of 0, 1, and every real number in between.

The following terms can describe a function $f(x)$, specifically how “well-behaved” f is.

- A *continuous* function f has the property that there is a function $h(x, \epsilon)$ (where x is in f ’s domain and $\epsilon > 0$), such that $f(x)$ and $f(y)$ are less than ϵ apart whenever x and y are in f ’s domain and less than $h(x, \epsilon)$ apart. Roughly speaking, for each x in f ’s domain, $f(x)$ and $f(y)$ are “close” if x and y are “close” and belong in the domain.
- If f is continuous, its *derivative*⁷ is the instantaneous rate of change in $f(\lambda)$ with respect to its variable λ . A derivative is, roughly speaking, a “slope” function or “velocity” function or “instantaneous-rate-of-change” function. The derivative (or *first derivative*) is denoted f' or $f^{(1)}$. The *second derivative* (“slope-of-slope”) of f , denoted f'' or $f^{(2)}$, is the derivative of f' ; the *third derivative*, denoted $f^{(3)}$, is the derivative of f'' ; and so on. The *0-th derivative* of a function f is f itself and denoted $f^{(0)}$.
- A *Hölder continuous*⁸ function (with M being the *Hölder constant* and α being the *Hölder exponent*) is a continuous function f such that $f(x)$ and $f(y)$ are no more than $M \delta^\alpha$ apart whenever x and y are in the function’s domain and no more than δ apart. Here, α satisfies $0 < \alpha \leq 1$. Roughly speaking, the function’s “steepness” is no greater than that of $M \cdot x^\alpha$.
- A *Lipschitz continuous* function with constant L (the *Lipschitz constant*) is Hölder continuous with Hölder exponent 1 and Hölder constant L . Roughly speaking, the function’s “steepness” is no greater than that of $L \cdot x$. If the function has a derivative on its domain, L can be the maximum of the absolute value of that derivative.
- A *convex* function f has the property that $f((x+y)/2) \leq (f(x) + f(y))/2$ whenever x , y , and $(x+y)/2$ are in the function’s domain. Roughly speaking, if the function’s “slope” never goes down, then it’s convex.

²<https://github.com/peteroupc/peteroupc.github.io/raw/master/bernsupp.md>

³<https://github.com/peteroupc/peteroupc.github.io/blob/master/bernsupp.md>

⁴<https://github.com/peteroupc/peteroupc.github.io/issues>

⁵<https://peteroupc.github.io/bernreq.html>

⁶<https://github.com/peteroupc/peteroupc.github.io/issues/18>

⁷<https://mathworld.wolfram.com/Derivative.html>

⁸https://en.wikipedia.org/wiki/Hölder_condition

- A *concave* function f has the property that $f((x+y)/2) \geq (f(x)+f(y))/2$ whenever x , y , and $(x+y)/2$ are in the function's domain. Roughly speaking, if the function's "slope" never goes up, then it's concave.
- A function f is *bounded* if there are two numbers a and b such that $a \leq f(x) \leq b$ whenever x is in the function's domain.

Note: The "good behavior" of a function can be important when designing Bernoulli factory algorithms. This page mostly cares how f behaves when its domain is the closed unit interval, that is, when $0 \leq x \leq 1$.

5 General Factory Functions

The algorithm for **general factory functions**⁹, described in my main article on Bernoulli factory algorithms, works by building randomized upper and lower bounds for a known function $f(\lambda)$, based on flips of the input coin. Roughly speaking, the algorithm works as follows:

1. Generate a random variate, U , uniformly distributed, greater than 0 and less than 1.
2. Flip the input coin, then build an upper and lower bound for $f(\lambda)$, based on the outcomes of the flips so far.
3. If U is less than or equal to the lower bound, return 1. If U is greater than the upper bound, return 0. Otherwise, go to step 2.

These randomized upper and lower bounds come from two sequences of polynomials as follows:

1. One sequence approaches the function $f(\lambda)$ from above, the other from below, and both sequences must converge to f .
2. For each sequence, the first polynomial has degree 1 (so is a linear function), and each other polynomial's degree is 1 higher than the previous.
3. The *consistency requirement* must be met, namely—
 - the difference between the degree- $(n-1)$ upper polynomial and the degree- n upper polynomial, and
 - the difference between the degree- n lower polynomial and the degree- $(n-1)$ lower polynomial,
must have nonnegative Bernstein coefficients, once each of these differences is rewritten as a polynomial of degree exactly n . (For more on Bernstein coefficients and the Bernstein form of polynomials, see "**Certain Polynomials**"¹⁰ in the main article.)

The consistency requirement ensures that the upper polynomials "decrease" and the lower polynomials "increase". Unfortunately, the reverse is not true in general; even if the upper polynomials "decrease" and the lower polynomials "increase" to f , this does not ensure the consistency requirement by itself.

Example (Nacu & Peres [2005]¹¹): The polynomial $x^2 + (1-x)^2$ is of degree 2 with Bernstein coefficients $[1, 0, 1]$. The polynomial $x(1-x)$ is of degree 2 with Bernstein coefficients $[0, 1/2, 0]$. Although $(x^2 + (1-x)^2)$ minus $(x(1-x))$ is nonnegative, this difference's Bernstein coefficients of degree 2 are not always nonnegative, namely, the Bernstein coefficients are $[1, -1/2, 1]$.

In this document, **fbelow**(n, k) and **fabove**(n, k) mean the k^{th} Bernstein coefficient for the lower or upper degree- n polynomial, respectively, where $0 \leq k \leq n$ is an integer.

The section "Building the Lower and Upper Polynomials" are ways to build sequences of polynomials that appropriately converge to a factory function if that function meets certain conditions.

To determine if the methods are right for $f(\lambda)$, a deep mathematical analysis of f is required; it would be helpful to plot that function using a computer algebra system to see if it is described in the next section.

⁹https://peteroupc.github.io/bernoulli.html#General_Factory_Functions

¹⁰https://peteroupc.github.io/bernoulli.html#Certain_Polynomials

¹¹Nacu, Șerban, and Yuval Peres. "Fast simulation of new coins from old", The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

5.1 Building the Lower and Upper Polynomials

The rest of this section assumes $f(\lambda)$ is not a constant. For examples of functions, see “**Examples of Well-Behaved Functions**”, in the appendix.

Concave functions. If f is concave, then **fbelow**(n, k) can equal $f(k/n)$, thanks to Jensen’s inequality.

Convex functions. If f is convex, then **fabove**(n, k) can equal $f(k/n)$, thanks to Jensen’s inequality.

Hölder and Lipschitz-continuous functions. I have found a way to extend the results of Nacu and Peres (2005)¹² to certain functions with a slope that tends to a vertical slope. The following scheme, proved in the appendix, implements **fabove** and **fbelow** if $f(\lambda)$ —

- is **Hölder continuous**¹³ on the closed unit interval, with Hölder constant m or less and Hölder exponent α (see “**Definitions**” as well as “**Examples of Well-Behaved Functions**”, in the appendix), and
- on the closed unit interval—
 - has a minimum of greater than 0 and a maximum of less than 1, or
 - is convex and has a minimum of greater than 0, or
 - is concave and has a maximum of less than 1.

For every integer n that’s a power of 2:

- $D(n) = m \cdot (2/7)^{\alpha/2} / ((2^{\alpha/2} - 1) \cdot n^{\alpha/2})$.
- **fbelow**(n, k) = $f(k/n)$ if f is concave; otherwise, $\min(\mathbf{fbelow}(4,0), \mathbf{fbelow}(4,1), \dots, \mathbf{fbelow}(4,4))$ if $n < 4$; otherwise, $f(k/n) - D(n)$.
- **fabove**(n, k) = $f(k/n)$ if f is convex; otherwise, $\max(\mathbf{fabove}(4,0), \mathbf{fabove}(4,1), \dots, \mathbf{fabove}(4,4))$ if $n < 4$; otherwise, $f(k/n) + D(n)$.

Notes:

1. If α is 1, $D(n)$ can be $m \cdot 322613 / (250000 \cdot \sqrt{n})$, which is an upper bound. If α is $1/2$, $D(n)$ can be $m \cdot 154563 / (40000 \cdot n^{1/4})$, which is an upper bound.
2. The function $f(x) = \min(\lambda t, 1 - \epsilon)$, where $\epsilon \geq 0$ and $t \geq 1$, is Lipschitz continuous with Lipschitz constant t . Because f is linear between 0 and $1/t$, ways to build polynomials that converge to this kind of function were discussed by Thomas and Blanchet (2012)^{14 15} and Nacu & Peres (2005)^{16 17}.

¹²Nacu, Şerban, and Yuval Peres. “**Fast simulation of new coins from old**”, The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aop/1106922322>

¹³https://en.wikipedia.org/wiki/Hölder_condition

¹⁴Thomas, A.C., Blanchet, J., “**A Practical Implementation of the Bernoulli Factory**”, arXiv:1106.2508v3 [stat.AP], 2012. <https://arxiv.org/abs/1106.2508v3>

¹⁵Thomas and Blanchet (2012) dealt with building polynomials that approach piecewise linear functions “fast”. Their strategy for $f(\lambda) = \min(mult \times \lambda, 1 - 2\epsilon)$ is to first compute a low-degree polynomial P satisfying $P(0) = 0$ and otherwise greater than f , and then compute further polynomials of increasing degree that each come between f and the previous polynomial and satisfy the consistency requirement. These polynomials approach f rapidly when λ is near 0, and extremely slowly when λ is near 1. In their strategy, **fbelow**(n, k) is $\min((k/n) \cdot mult, 1 - \epsilon)$, and **fabove**(n, k) is $\min((k/n) \cdot y/x, y)$, where: (1): $x = -((y - (1 - \epsilon)) / \epsilon)^5 / mult + y / mult$. (This formula doesn’t appear in their paper, but in the **supplemental source code** uploaded by A. C. Thomas after I asked for that source code.) (2): y satisfies $0 < y < 1$ and is chosen so that the degree- n polynomial is between f and the previous polynomial and meets the consistency requirement. The supplemental source code seems to choose y in an *ad hoc* manner. (3): n is the polynomial’s degree. <https://github.com/acthomasca/rberfac/blob/main/rberfac-public-2.R>

¹⁶Nacu, Şerban, and Yuval Peres. “**Fast simulation of new coins from old**”, The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aop/1106922322>

¹⁷In Nacu and Peres (2005), the following polynomial sequences were suggested to simulate $\min(2\lambda, 1 - 2\epsilon)$ (using the algorithms from the section “General Factory Functions” in “**Bernoulli Factory Algorithms**”), provided $\epsilon < 1/8$, where n is a power of 2. However, with these sequences, each simulation will require an extraordinary number of input coin flips. **fbelow**(n, k) = $\min(2(k/n), 1 - 2\epsilon)$. **fabove**(n, k) = $\min(2(k/n), 1 - 2\epsilon) + \frac{2 \times \max(0, k/n + 3\epsilon - 1/2)}{\epsilon(2 - \sqrt{2})} \sqrt{2/n} + \frac{72 \times \max(0, k/n - 1/9)}{1 - \exp(-2 \times \epsilon^2)} \exp(-2n \times \epsilon^2)$. <https://peteroupc.github.io/bernoulli.html>

Functions with a Lipschitz-continuous derivative. The following method, proved in the appendix, implements **fabove** and **fbelow** if $f(\lambda)$ —

- has a Lipschitz-continuous derivative (see “**Definitions**” as well as “**Examples of Well-Behaved Functions**”, in the appendix), and
- in the closed unit interval—
 - has a minimum of greater than 0 and a maximum of less than 1, or
 - is convex and has a minimum of greater than 0, or
 - is concave and has a maximum of less than 1.

Let m be the Lipschitz constant of f ’s derivative, or a greater number than that constant. Then for every integer n that’s a power of 2:

- For every n such that **fbelow**(n, k) is 0 or greater for every k , **fbelow**(n, k) = $f(k/n)$ if f is concave; otherwise, $\min(\mathbf{fbelow}(4,0), \mathbf{fbelow}(4,1), \dots, \mathbf{fbelow}(4,4))$ if $n < 4$; otherwise, $f(k/n) - m/(7*n)$. For every other n , **fbelow**(n, k) = 0.
- For every n such that **fabove**(n, k) is 1 or less for every k , **fabove**(n, k) = $f(k/n)$ if f is convex; otherwise, $\max(\mathbf{fabove}(4,0), \mathbf{fabove}(4,1), \dots, \mathbf{fabove}(4,4))$ if $n < 4$; otherwise, $f(k/n) + m/(7*n)$. For every other n , **fabove**(n, k) = 1.

Certain functions that equal 0 at 0. This approach involves transforming the function f so that it no longer equals 0 at the point 0. This can be done by dividing f by a function (**High**(λ)) that “dominates” f everywhere on the closed unit interval. Unlike for the original function, there might be a polynomial-building scheme described earlier in this section for the transformed function.

More specifically, **High**(λ) must meet the following requirements:

- **High**(λ) is continuous on the closed unit interval.
- **High**(0) = 0. (This is required to ensure correctness in case λ is 0.)
- $1 \geq \mathbf{High}(1) \geq f(1) \geq 0$.
- $1 > \mathbf{High}(\lambda) > f(\lambda) > 0$ whenever $0 < \lambda < 1$.
- If $f(1) = 0$, then **High**(1) = 0. (This is required to ensure correctness in case λ is 1.)

Also, **High** is a Bernoulli factory function that should admit a simple Bernoulli factory algorithm. For example, **High** can be the following degree- n polynomial: $1 - (1 - \lambda)^n$, where n is a positive integer.¹⁸

The algorithm is now described.

Let $g(\lambda) = \lim_{\nu \rightarrow \lambda} f(\nu)/\mathbf{High}(\nu)$ (roughly speaking, the value that $f(\nu)/\mathbf{High}(\nu)$ approaches as ν approaches λ .) If—

- $f(0) = 0$ and $f(1) < 1$, and
- $g(\lambda)$ is continuous on the closed unit interval and belongs in one of the classes of functions given earlier,

then f can be simulated using the following algorithm:

1. Run a Bernoulli factory algorithm for **High**. If the call returns 0, return 0. (For example, if **High**(λ) = λ , then this step amounts to the following: “Flip the input coin. If it returns 0, return 0.”)
2. Run a Bernoulli factory algorithm for $g(\cdot)$ and return the result of that algorithm. This can be one of the **general factory function algorithms**¹⁹ if there is a way to calculate polynomials that converge to $g(\cdot)$ in a manner needed for that algorithm (for example, if g is described earlier in this section).

Notes:

1. It may happen that $g(0) = 0$. In this case, step 2 of this algorithm can involve running this algorithm again, but with new g and **High** functions that are found based on the current g

¹⁸In this case, an algorithm to simulate **High**(λ) is: Flip the input coin n times or until a flip returns 1, whichever comes first, then output the last coin flip result.

¹⁹https://peteroupc.github.io/bernoulli.html#General_Factory_Functions

function. (This will eventually result in $g(0)$ being nonzero if f is a nonconstant Bernoulli factory function.) See the second example below.

2. $\text{High}(\lambda)$ can also equal 1 instead of be described in this section. That leads to the original Bernoulli factory algorithm for $f(\lambda)$.

Examples:

1. If $f(\lambda) = (\sinh(\lambda) + \cosh(\lambda) - 1)/4$, then f is less than or equal to $\text{High}(\lambda) = \lambda$, so $g(\lambda)$ is $1/4$ if $\lambda = 0$, and $(\exp(\lambda) - 1)/(4 * \lambda)$ otherwise. The following code in Python that uses the SymPy computer algebra library computes this example: `fx = (sinh(x)+cosh(x)-1)/4; h = x; pprint(Piecewise((limit(fx/h,x,0), Eq(x,0)), ((fx/h).simplify(), True)))`.
2. If $f(\lambda) = \cosh(\lambda) - 1$, then f is less than or equal to $\text{High}(\lambda) = \lambda$, so $g(\lambda)$ is 0 if $\lambda = 0$, and $(\cosh(\lambda) - 1)/\lambda$ otherwise. Now, since $g(0) = 0$, find new functions g and High based on the current g . The current g is less than or equal to $\text{High}(\lambda) = \lambda * 3 * (2 - \lambda)/5$ (a degree-2 polynomial that has Bernstein coefficients $[0, 6/10, 6/10]$), so $G(\lambda) = 5/12$ if $\lambda = 0$, and $-(5 * \cosh(\lambda) - 5)/(3 * \lambda^2 * (\lambda - 2))$ otherwise. G is bounded away from 0 and 1, resulting in the following algorithm:
 1. (Simulate High .) Flip the input coin. If it returns 0, return 0.
 2. (Simulate High .) Flip the input coin twice. If both flips return 0, return 0. Otherwise, with probability $4/10$ (that is, 1 minus $6/10$), return 0.
 3. Run a Bernoulli factory algorithm for G (which might involve building polynomials that converge to G , noticing that G 's derivative is Lipschitz continuous) and return the result of that algorithm.

Certain functions that equal 0 at 0 and 1 at 1. Let f , g , and High be functions as defined earlier, except that $f(0) = 0$ and $f(1) = 1$. Define the following additional functions:

- $\text{Low}(\lambda)$ is a function that meets the following requirements:
 - $\text{Low}(\lambda)$ is continuous on the closed unit interval.
 - $\text{Low}(0) = 0$ and $\text{Low}(1) = 1$.
 - $1 > f(\lambda) > \text{Low}(\lambda) > 0$ whenever $0 < \lambda < 1$.
- $q(\lambda) = \lim_{\nu \rightarrow \lambda} \text{Low}(\nu)/\text{High}(\nu)$.
- $r(\lambda) = \lim_{\nu \rightarrow \lambda} (1 - g(\nu))/(1 - q(\nu))$.

Roughly speaking, Low is a function that bounds f from below, just as High bounds f from above. Low is a Bernoulli factory function that should admit a simple Bernoulli factory algorithm; one example is the polynomial λ^n where n is a positive integer. If both Low and High are polynomials of the same degree, q will be a ratio of polynomials with a relatively simple Bernoulli factory algorithm (see “**Certain Rational Functions**”²⁰).

Now, if $r(\lambda)$ is continuous on the closed unit interval, then f can be simulated using the following algorithm:

1. Run a Bernoulli factory algorithm for High . If the call returns 0, return 0. (For example, if $\text{High}(\lambda) = \lambda$, then this step amounts to the following: “Flip the input coin. If it returns 0, return 0.”)
2. Run a Bernoulli factory algorithm for $q(\cdot)$. If the call returns 1, return 1.
3. Run a Bernoulli factory algorithm for $r(\cdot)$, and return 1 minus the result of that call. The Bernoulli factory algorithm can be one of the **general factory function algorithms**²¹ if there is a way to calculate polynomials that converge to $r(\cdot)$ in a manner needed for that algorithm (for example, if r is described earlier in this section).

Notes:

²⁰https://peteroupc.github.io/bernoulli.html#Certain_Rational_Functions

²¹https://peteroupc.github.io/bernoulli.html#General_Factory_Functions

1. Quick proof: Rewrite $f = \text{High} \cdot (q \cdot 1 + (1 - q) \cdot (1 - r)) + (1 - \text{High}) \cdot 0$.
2. $\text{High}(\lambda)$ is allowed to equal 1 if the $r(\cdot)$ in step 3 is allowed to equal 0 at 0.

Example: If $f(\lambda) = (1 - \exp(\lambda))/(1 - \exp(1))$, then f is less than or equal to $\text{High}(\lambda) = \lambda$, and greater than or equal to $\text{Low}(\lambda) = \lambda^2$. As a result, $q(\lambda) = \lambda$, and $r(\lambda) = (2 - \exp(1))/(1 - \exp(1))$ if $\lambda = 0$; $1/(\exp(1) - 1)$ if $\lambda = 1$; and $(-\lambda \cdot (1 - \exp(1)) - \exp(\lambda) + 1)/(\lambda \cdot (1 - \exp(1)) \cdot (\lambda - 1))$ otherwise. This can be computed using the following code in Python that uses the SymPy computer algebra library: `fx=(1-exp(x))/(1-exp(1)); high=x; low=x**2; q=(low/high); r=(1-fx/high)/(1-q); r=Piecewise((limit(r, x, 0), Eq(x,0)), (limit(r,x,1),Eq(x,1)), (r,True)).simplify(); pprint(r).`

Other functions that equal 0 or 1 at the endpoint 0, 1, or both. The table below accounts for these Bernoulli factory functions:

If $f(0) =$	And $f(1) =$	Method
> 0 and < 1	1	Use the algorithm for certain functions that equal 0 at 0 , but with $f(\lambda) = 1 - f(1 - \lambda)$. <i>Inverted coin:</i> Instead of the usual input coin, use a coin that does the following: “Flip the input coin and return 1 minus the result.” <i>Inverted result:</i> If the overall algorithm would return 0, it returns 1 instead, and vice versa.
> 0 and < 1	0	Algorithm for certain functions that equal 0 at 0 , but with $f(\lambda) = f(1 - \lambda)$. (For example, $\cosh(\lambda) - 1$ becomes $\cosh(1 - \lambda) - 1$.) <i>Inverted coin.</i>
1	0	Algorithm for certain functions that equal 0 at 0 and 1 at 1 , but with $f(\lambda) = 1 - f(\lambda)$. <i>Inverted result.</i>
1	> 0 and ≤ 1	Algorithm for certain functions that equal 0 at 0 , but with $f(\lambda) = 1 - f(\lambda)$. <i>Inverted result.</i>

5.2 Another General Algorithm

The algorithm I’ve developed in this section simulates $f(\lambda)$ when f belongs in a large class of functions, as long as the following is known:

- f is continuous and has a minimum of greater than 0 and a maximum of less than 1.
- There is a family of polynomials ($L_1(f)$, $L_2(f)$, $L_4(f)$, $L_8(f)$, ...) that come close to f with a known error bound, where the number after L is the degree of the polynomial.
- There is a way to find the *Bernstein coefficients* of each polynomial $L_n(f)$ in the family of polynomials.

For examples of suitable polynomials, see “**Approximations in Bernstein Form**”²².

In effect, the algorithm writes f as an infinite sum of polynomials, whose maximums must sum to 1 or less (called T in the algorithm below), then simulates an appropriate **convex combination**²³ (weighted sum) of these polynomials. To build the convex combination, each polynomial in the infinite sum is divided by an upper bound on its maximum (which is why error bounds on $L_n(f)$ are crucial here).²⁴ To simulate f , the algorithm—

- selects a polynomial in the convex combination with probability proportional to its upper bound, or a “leftover” zero polynomial with probability T , then
- simulates the chosen polynomial (which is easy to do; see Goyal and Sigman (2012)²⁵).

²²<https://peteroupc.github.io/bernapprox.html>

²³https://peteroupc.github.io/bernoulli.html#Convex_Combinations

²⁴With this infinite sum and these upper bounds, Lemma 4 of Holtz et al. (2011) says that a Bernoulli factory algorithm for $f(\lambda)$ is possible.

²⁵Vineet Goyal and Karl Sigman. 2012. On simulating a class of Bernstein polynomials. ACM Trans. Model. Comput. Simul. 22, 2, Article 12 (March 2012), 5 pages. <https://doi.org/10.1145/2133390.2133396>

-
- In the algorithm, denote:
 - $\epsilon(f, n)$ as an upper bound on the absolute value of the difference between f and the degree- n polynomial $L_n(f)$. $\epsilon(f, n)$ must increase nowhere as n increases, and must converge to 0.
 - * For best results, this should be written as $\epsilon(f, n) = C/n^r$, where C is a constant and $r > 0$ is a multiple of $1/2$, since then it's easy to find the value of $\text{ErrShift}(f, n)$, later. In this case, the algorithm should be limited to functions with a continuous $(2r)$ -th derivative or a Lipschitz-continuous $(2r - 1)$ -th derivative (see “**Achievable Simulation Rates**”, later). (For example, if the error bound is C/n^2 , the function f should have a continuous fourth derivative or a Lipschitz-continuous third derivative.)
 - * For examples of error bounds, see “**Approximations in Bernstein Form**”²⁶.
 - $\text{ErrShift}(f, m)$ as $1.01 \cdot \sum_{i \geq m} \epsilon(f, 2^i)$. The factor 1.01 is needed to ensure each difference polynomial is strictly between 0 and 1.
 - * **Example:** If $\epsilon(f, n) = C/n^r$, then $\text{ErrShift}(f, m) = 1.01 \cdot C \cdot 2^r / (((2^r) - 1) \cdot 2^{rm})$.
 - $\text{DiffWidth}(f, m)$ as $1.01 \cdot 2(\epsilon(f, 2^m) + \epsilon(f, 2^{m+1}))$. This is an upper bound on the maximum difference between the shifted degree- 2^m and the shifted degree- (2^{m+1}) polynomial.
 - The technique breaks f into a **starting polynomial** and a family of **difference polynomials**. To find the **starting polynomial**:
 1. Set m to 0.
 2. Find the Bernstein coefficients of L_{2^m} , then subtract $\text{ErrShift}(f, m)$ from them. If those coefficients now all lie in the closed unit interval, go to the next step. Otherwise, add 1 to m and repeat this step.
 3. Calculate **StartWidth** as $\text{ceil}(c \cdot 65536) / 65536$, where c is the maximum Bernstein coefficient from step 2, then divide each Bernstein coefficient by **StartWidth**. (65536 is arbitrary and ensures **StartWidth** is a rational number that is close to, but no lower than, the maximum Bernstein coefficient, for convenience.)
 4. The **starting polynomial** now has Bernstein coefficients found in step 3. Set **StartOrder** to m .
 - To find the **difference polynomial** of order m :
 1. Find the Bernstein coefficients of L_{2^m} , then subtract $\text{ErrShift}(f, m)$ from them. Rewrite them to Bernstein coefficients of degree 2^{m+1} . Call the coefficients **LowerCoeffs**.
 2. Find the Bernstein coefficients of $L_{2^{m+1}}$, then subtract $\text{ErrShift}(f, m + 1)$ from them. Call the resulting values **UpperCoeffs**.
 3. Subtract **UpperCoeffs** from **LowerCoeffs**, and call the result **DiffCoeffs**. Divide each coefficient in **DiffCoeffs** by $\text{DiffWidth}(f, m)$. The result is the Bernstein coefficients of a positive polynomial of degree 2^{m+1} bounded by 0 and 1, but these coefficients are not necessarily bounded by 0 and 1. Thus, if any coefficient in **DiffCoeffs** is less than 0 or greater than 1, add 1 to m and rewrite **DiffCoeffs** to Bernstein coefficients of degree 2^{m+1} until no coefficient is less than 0 or greater than 1 anymore.
 4. The **difference polynomial** now has Bernstein coefficients given by **DiffCoeffs**.
 - The probabilities for X are as follows:
 - First, find the **starting polynomial**, then calculate T as $\text{StartWidth} + \sum_{i \geq 0} \text{DiffWidth}(f, \text{StartOrder} + i)$. If T is greater than 1, this algorithm can't be used.
 - * **Example:** If $\epsilon(f, n) = C/n^r$, then $T = \text{StartWidth} + 1.01 \cdot (2 \cdot 2^{-r \cdot \text{StartOrder}} C (1 + 2^{-r})) / (1 - 2^{-r})$.
 - X is 0 with probability $1 - T$.
 - X is 1 with probability equal to **StartWidth**.
 - For each $m \geq 2$, X is m with probability equal to $\text{DiffWidth}(f, \text{StartOrder} + m - 2)$.

Then an algorithm to toss heads with probability equal to f would be:

1. Generate X at random with the probabilities given earlier.

²⁶<https://peteroupc.github.io/bernapprox.html>

2. If X is 0, return 0. Otherwise, if X is 1, find the **starting polynomial** and its Bernstein coefficients. Otherwise (if X is 2 or greater), find the **difference polynomial** of order m and its Bernstein coefficients, where $m = (X - 2) + \text{StartOrder}$.
3. Flip the input coin (with probability of heads λ), $n - 1$ times, where n is the number of Bernstein coefficients in the polynomial found in step 2 (its degree plus one), and let j be the number of heads.
4. Return 1 with probability equal to the polynomial's j th Bernstein coefficient (j starts at 0), or 0 otherwise (see also Goyal and Sigman 2012²⁷ for an algorithm to simulate polynomials).

If T turns out to be greater than 1 in this algorithm, but still finite, one way to simulate f is to create a coin that simulates $f(\lambda)/T$ instead, and use that coin as the input to a **linear Bernoulli factory**²⁸ that simulates $T \cdot (f(\lambda)/T)$. (This is possible especially because $f(\lambda)$ is assumed to have a maximum of less than 1.)

Example: The following parameters allow this algorithm to work if f is concave, has a maximum of less than 1, and has a Lipschitz-continuous derivative with Lipschitz constant M or less. In this case, it is allowed that $f(0) = 0$, $f(1) = 0$, or both.

- The family of polynomials $L_n(f)$ is simply the family of Bernstein polynomials of f . The Bernstein coefficients of $L_n(f)$ are $f(0/n), f(1/n), \dots, f(n/n)$.
- The error bound is $\epsilon(f, n) = M/(8n)$ (Lorentz 1964)²⁹.
- The **starting polynomial** is found as follows. Let $c = \max(f(0), f(1))$. Then the starting polynomial has two Bernstein coefficients both equal to c ; **StartWidth** is equal to $\text{ceil}(c \cdot 65536)/65536$, and **StartOrder** is equal to 0.
- $\text{ErrShift}(f, m) = 0$. The reason for 0 is that f is concave, so its Bernstein polynomials naturally “increase” with increasing degree (Temple 1954)³⁰, (Moldovan 1962)³¹.
- $\text{DiffWidth}(f, m) = 1.01 \cdot 3M/(8 \cdot 2^m)$. For the same reason as the previous point, and because the Bernstein polynomials are always “below” f , $\text{DiffWidth}(f, m)$ can also equal $1.01 \cdot \epsilon(f, 2^m) = 1.01 \cdot M/(8 \cdot 2^m)$. This is what is used to calculate T , later.
- T is calculated as **StartWidth** + $1.01 \cdot M/4$.

5.3 Request for Additional Methods

Readers are requested to let me know of additional solutions to the following problem:

*Let $f(\lambda)$ be continuous and satisfy $0 < f(\lambda) < 1$ whenever $0 \leq \lambda \leq 1$. Given that $f(\lambda)$ belongs to a large class of functions (for example, it has a continuous, Lipschitz continuous, concave, or nowhere decreasing k -th derivative for some integer k , or any combination of these), compute the Bernstein coefficients for two sequences of polynomials as follows: one of them approaches $f(\lambda)$ from above, the other from below, and the consistency requirement must be met (see “**General Factory Functions**”).*

The polynomials need to be computed only for degrees 2, 4, 8, 16, and higher powers of 2.

The rate of convergence must be no slower than $1/n^{r/2}$ if the specified class has only functions with continuous r -th derivative.

²⁷Vineet Goyal and Karl Sigman. 2012. On simulating a class of Bernstein polynomials. ACM Trans. Model. Comput. Simul. 22, 2, Article 12 (March 2012), 5 pages. <https://doi.org/10.1145/2133390.2133396>

²⁸https://peteroupc.github.io/bernoulli.html#Linear_Bernoulli_Factories

²⁹Lorentz, G.G. (1964). Inequalities and the Saturation Classes of Bernstein Polynomials. In: Butzer, P.L., Korevaar, J. (eds) On Approximation Theory / Über Approximationstheorie. ISNM International Series of Numerical Mathematics / Internationale Schriftenreihe zur Numerischen Mathematik / Série Internationale D’Analyse Numérique, vol 5 . Springer, Basel. https://doi.org/10.1007/978-3-0348-4131-3_19

³⁰Temple, W.B., “Stieltjes integral representation of convex functions”, Duke Math. J. 21(3): 527-531 (September 1954). <https://doi.org/10.1215/S0012-7094-54-02152-3>

³¹Moldovan, E., “Observations sur la suite des polynômes de S. N. Bernstein d’une fonction continue”, Mathematica (Cluj) 4.27 (1962): 289-292.

Methods that use only integer arithmetic and addition and multiplication of rational numbers are preferred (thus, methods that involve cosines, sines, π , $\exp$, and $\ln$ are not preferred).

See also the **open questions**³².

6 Approximate Bernoulli Factories

An **approximate Bernoulli factory** for a function $f(\lambda)$ is a Bernoulli factory algorithm that simulates another function, $g(\lambda)$, that approximates f in some sense.

Usually g is a polynomial, but can also be a rational function (ratio of polynomials) or another function with an easy-to-implement Bernoulli factory algorithm.

Meanwhile, $f(\lambda)$ can be any function that maps the closed unit interval to itself, even if it isn't continuous or a factory function (examples include the “step function” 0 if $\lambda < 1/2$ and 1 otherwise, or the function $2 \cdot \min(\lambda, 1 - \lambda)$). If the function is continuous, it can be approximated arbitrarily well by an approximate Bernoulli factory (as a result of the so-called “Weierstrass approximation theorem”), but generally not if the function is discontinuous.

To build an approximate Bernoulli factory with a polynomial:

1. First, find a polynomial in Bernstein form of degree n that is close enough to the desired function $f(\lambda)$.

The simplest choice for this polynomial, known simply as a *Bernstein polynomial*, has $n+1$ Bernstein coefficients and its j^{th} coefficient (starting at 0) is found as $f(j/n)$. For this choice, if f is continuous, the polynomial can be brought arbitrarily close to f by choosing n high enough.

Other choices for this polynomial are given in the page “**Approximations in Bernstein Form**”³³.

Whatever polynomial is used, the polynomial's Bernstein coefficients must all lie on the closed unit interval.

The polynomial can be in *homogeneous form* (also known as *scaled Bernstein form* (Farouki and Rajan 1988)³⁴) instead of in Bernstein form, with *scaled Bernstein coefficients* $s_0, \dots, s_n$, as long as $0 \leq s_i \leq \binom{n}{i}$ where $0 \leq i \leq n$.

2. The rest of the process is to toss heads with probability equal to that polynomial, given its Bernstein coefficients. To do this, first flip the input coin n times, and let j be the number of times the coin returned 1 this way.
3. Then, with probability equal to—
 - the polynomial's Bernstein coefficient at position j (which will be $f(j/n)$ in the case of the Bernstein polynomial of f), or
 - the polynomial's scaled Bernstein coefficient at position j , divided by $\text{choose}(n, j)$,

return 1. Otherwise, return 0. Here, $0 \leq j \leq n$.

If the probability can be an irrational number, see “**Algorithms for General Irrational Constants**”³⁵ for ways to exactly sample a probability equal to that irrational number.

Notes:

³²https://peteroupc.github.io/bernreq.html#Polynomials_that_approach_a_factory_function_fast

³³<https://peteroupc.github.io/bernapprox.html>

³⁴Farouki, Rida T., and V. T. Rajan. “Algorithms for polynomials in Bernstein form”. Computer Aided Geometric Design 5, no. 1 (1988): 1-26. <https://doi.org/10.1016/0167-8396%2888%2990016-7>
<https://www.sciencedirect.com/science/article/pii/0167839688900167>

³⁵https://peteroupc.github.io/bernoulli.html#Algorithms_for_General_Irrational_Constants

1. More sophisticated ways to implement steps 2 and 3 are found in the section “**Certain Polynomials**³⁶” in the main article “Bernoulli Factory Algorithms”.
2. There are other kinds of functions, besides polynomials and rational functions, that serve to approximate continuous functions. But many of them work poorly as approximate Bernoulli factory functions because their lack of “smoothness” means there is no simple Bernoulli factory for them. For example, a *spline*, which is a continuous function made up of a finite number of polynomial pieces, is generally not “smooth” at the points where the spline’s pieces meet.
3. Bias and variance are the two sources of error in a randomized estimation algorithm. Let $g(\lambda)$ be an approximation of $f(\lambda)$. The original Bernoulli factory for f , if it exists, has bias 0 and variance $f(\lambda)(1 - f(\lambda))$, but the approximate Bernoulli factory has bias $g(\lambda) - f(\lambda)$ and variance $g(\lambda)(1 - g(\lambda))$. (“Variance reduction” methods are outside the scope of this document.) An estimation algorithm’s *mean squared error* equals variance plus square of bias.
4. There are two known approximations to the linear function $f(\lambda) = 2\lambda$ using a polynomial in Bernstein form of degree n that maps the open interval $(0, 1)$ to itself. In each case, if $g(\lambda)$ is that polynomial and if $0 \leq \lambda \leq 1/2$, then the error in approximating $f(\lambda)$ is no greater than $1 - g(1/2)$.
 - In Henderson and Glynn (2003, Remark 4)³⁷, the polynomial’s j^{th} Bernstein coefficient (starting at 0) is $\min((j/n)^2, 1 - 1/n)$. The polynomial g can be computed with the SymPy computer algebra library as follows: `from sympy.stats import *; g=2*E( Min(sum(Bernoulli(("B%d" % (i)),z) for i in range(n))/n, (S(1)-S(1)/n)/2))`.
 - In Nacu and Peres (2005, section 6)³⁸, the polynomial’s j^{th} Bernstein coefficient (starting at 0) is $\min((j/i)^2, 1)$. It corresponds to the following algorithm: Flip the input coin n times or until the ratio of “heads” to “flips” becomes at least $1/2$, whichever comes first, then if n flips were made without the ratio becoming at least $1/2$, return 0; otherwise, return 1.

7 Achievable Simulation Rates

In general, the number of input coin flips needed by any Bernoulli factory algorithm for a factory function $f(\lambda)$ depends on how “smooth” the function f is.

The following table summarizes the rate of simulation (in terms of the number of input coin flips needed) that can be achieved *in theory* depending on $f(\lambda)$, assuming the input coin’s probability of heads is unknown. In the following table:

- λ , the unknown probability of heads, is ε or greater and $(1 - \varepsilon)$ or less for some $\varepsilon > 0$.
- The simulation makes use of a *fair coin* (a coin that shows 1 or 0 with equal probability) in addition to input coin flips.
- $\Delta(n, r, \lambda) = \max(\sqrt{\lambda(1 - \lambda)/n}, 1/n)^r$.

³⁶<https://peteroup.github.io/bernoulli.html>

³⁷Henderson, S.G., Glynn, P.W., “Nonexistence of a class of variate generation schemes”, *Operations Research Letters* 31 (2003). <https://doi.org/10.1016/S0167-6377%2802%2900217-1> . <https://www.sciencedirect.com/science/article/pii/S0167637702002171>

³⁸Nacu, Șerban, and Yuval Peres. “Fast simulation of new coins from old”, *The Annals of Applied Probability* 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

Property of simulation	Property of f
Requires no more than n input coin flips.	If and only if f can be written as a polynomial of degree n with Bernstein coefficients in the closed unit interval (Goyal and Sigman 2012) ³⁹ .
Requires a finite number of flips on average. Also known as “realizable” by Flajolet et al. (2010) ⁴⁰ .	Only if f is Lipschitz continuous (Nacu and Peres 2005) ⁴¹ . Whenever f admits a fast simulation (Mendo 2019) ⁴² .
Number of flips required, raised to power of r , is bounded by a finite number on average and has a tail that drops off uniformly over f ’s domain.	Only if f has continuous r -th derivative (Nacu and Peres 2005) ⁴³ .
Requires more than n flips with at most a probability proportional to $\Delta(n, r + 1, \lambda)$, for integer $r \geq 0$ and every λ , and for large enough n . (The greater r is, the faster the simulation.)	Only if f has an r -th derivative that is continuous and in the Zygmund class (see note later) (Holtz et al. 2011, Theorem 13) ⁴⁴ .
Requires more than n flips with at most a probability proportional to $\Delta(n, \alpha, \lambda)$, for noninteger $\alpha > 0$ and every λ , and for large enough n . (The greater α is, the faster the simulation.)	If and only if f has an r -th derivative that is Hölder continuous with Hölder exponent $(\alpha - r)$ or greater, where $r = \text{floor}(\alpha)$ (Holtz et al. 2011, Theorem 8) ⁴⁵ . Assumes f is bounded away from 0 and 1.
“Fast simulation” (requires more than n flips with a probability that decays exponentially or faster as n gets large). Also known as “strongly realizable” by Flajolet et al. (2010) ⁴⁶ .	If and only if f is analytic at every point in its domain (see note later) (Nacu and Peres 2005) ⁴⁷ .

³⁹Vineet Goyal and Karl Sigman. 2012. On simulating a class of Bernstein polynomials. *ACM Trans. Model. Comput. Simul.* 22, 2, Article 12 (March 2012), 5 pages. <https://doi.org/10.1145/2133390.2133396>

⁴⁰Flajolet, P., Pelletier, M., Soria, M., “On Buffon machines and numbers”, arXiv:0906.5560 [math.PR], 2010. <https://arxiv.org/abs/0906.5560>

⁴¹Nacu, Șerban, and Yuval Peres. “Fast simulation of new coins from old”, *The Annals of Applied Probability* 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

⁴²Mendo, Luis. “An asymptotically optimal Bernoulli factory for certain functions that can be expressed as power series.” *Stochastic Processes and their Applications* 129, no. 11 (2019): 4366-4384. <https://doi.org/10.1016/j.spa.2018.11.017> <https://www.sciencedirect.com/science/article/pii/S0304414918306768>

⁴³Nacu, Șerban, and Yuval Peres. “Fast simulation of new coins from old”, *The Annals of Applied Probability* 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

⁴⁴Holtz, O., Nazarov, F., Peres, Y., “New Coins from Old, Smoothly”, *Constructive Approximation* 33 (2011). <https://link.springer.com/content/pdf/10.1007/s00365-010-9108-5.pdf>

⁴⁵Holtz, O., Nazarov, F., Peres, Y., “New Coins from Old, Smoothly”, *Constructive Approximation* 33 (2011). <https://link.springer.com/content/pdf/10.1007/s00365-010-9108-5.pdf>

⁴⁶Flajolet, P., Pelletier, M., Soria, M., “On Buffon machines and numbers”, arXiv:0906.5560 [math.PR], 2010. <https://arxiv.org/abs/0906.5560>

⁴⁷Nacu, Șerban, and Yuval Peres. “Fast simulation of new coins from old”, *The Annals of Applied Probability* 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

⁴⁸Mendo, Luis. “An asymptotically optimal Bernoulli factory for certain functions that can be expressed as power series.” *Stochastic Processes and their Applications* 129, no. 11 (2019): 4366-4384. <https://doi.org/10.1016/j.spa.2018.11.017> <https://www.sciencedirect.com/science/article/pii/S0304414918306768>

Average number of flips greater than λ Whenever f admits a fast simulation (Mendo 2019)⁴⁸.
or equal to $(f'(\lambda))^2 \lambda^*(1 - \lambda)/(f(\lambda)^*(1 - f(\lambda)))$, where f' is the first derivative of f .

Note: A function $f(\lambda)$ is:

- *Analytic* at a point z if there is a positive number r such that f is writable as a *Taylor series* centered at z , namely—

$$f(\lambda) = f(z) + f^{(1)}(z)(\lambda - z)^1/1! + f^{(2)}(z)(\lambda - z)^2/2! + \dots,$$

for every point λ satisfying $\text{abs}(\lambda - z) < r$, where $f^{(i)}$ is the i -th derivative of f .

- In the *Zygmund class* if it is continuous and there is a positive number D with the following property: For each step size $\epsilon > 0$, $\text{abs}(f(x - h) + f(x + h) - 2f(x)) \leq D \times \epsilon$ wherever the left-hand side is defined and $0 < h \leq \epsilon$. The Zygmund class includes the two “smaller” classes of Lipschitz-continuous functions (see “Definitions”) and functions with a continuous derivative.

8 Notes

9 Appendix

9.1 Examples of Well-Behaved Functions

In the examples given in this section, $f(\lambda)$ is a function defined on the closed unit interval.

Generally, how well-behaved a function is depends on how many continuous derivatives it has, and whether those derivatives are Lipschitz continuous or Hölder continuous, among other things. The following lists several classes of functions, from worst to best behaved and from “largest” to “smallest”:

Functions continuous except possibly at one point $\rightarrow$ Continuous functions $\rightarrow$ Hölder-continuous functions $\rightarrow$ Lipschitz-continuous functions $\rightarrow$ With continuous first derivative $\rightarrow$ With Hölder continuous first derivative $\rightarrow$ With Lipschitz continuous first derivative $\rightarrow$ With continuous second derivative $\rightarrow$ With infinitely many derivatives $\rightarrow$ Analytic functions $\rightarrow$ Polynomials.

Concave and convex functions. The following table shows examples of functions that are convex, concave, or neither. All these functions are continuous. Also, review the **definitions**.

Function $f(\lambda)$	Convex or concave?	Notes
$1 - \lambda^2$.	Concave.	
λ^2 .	Convex.	
$\lambda^2 - \lambda^3$.	Neither.	
λ^z , where $0 < z \leq 1$.	Concave.	
λ^z , where $z \geq 1$.	Convex.	
$\exp(-\lambda/4)$.	Concave.	

Hölder- and Lipschitz-continuous functions. The following table shows some functions that are Hölder continuous and some that are not. Also, review the **definitions**.

Function $f(\lambda)$:	Hölder exponent (α) and an upper bound of the Hölder constant (L):	Notes
$\lambda^z \cdot t$.	$\alpha = z, L = \text{abs}(t)$.	$0 < z \leq 1, t \neq 0$.
$\lambda^z \cdot t$.	$\alpha = 1$ (Lipschitz continuous). $L = z \cdot \text{abs}(t)$.	$z \geq 1, t$ is a real number.
$\lambda^{1/3}/4 + \lambda^{2/3}/5$.	$\alpha = 1/3, L = 9/20$.	α is the minimum of Hölder exponents, $\min(1/3, 2/3)$, and L is the sum of Hölder constants, $1/4 + 1/5$.
$1/2 - (1 - 2\lambda)^z/2$ if $\lambda < 1/2$ and $1/2 + (2\lambda - 1)^z/2$ otherwise.	$\alpha = z, L = 2^z/2$.	$0 < z \leq 1$. In this example, f has a “vertical” slope at $1/2$, unless z is 1.
$3/4 - \sqrt{\lambda(1-\lambda)}$. Continuous and piecewise linear .	$\alpha = 1/2, L = 1$. $\alpha = 1, L$ is the greatest absolute value of the slope among all pieces’ slopes.	Has a “vertical” slope at 0 and 1. $f(\lambda)$ is <i>piecewise linear</i> if it’s made up of multiple linear functions defined on a finite number of “pieces”, or nonempty subintervals, that together make up f ’s domain (in this case, the closed unit interval).
Piecewise linear; equals 0 at 0, $3/4$ at $2/3$ and $1/4$ at 1, and these points are connected by linear functions.	$\alpha = 1, L = 1.5$.	$L = \max(\text{abs}((3/4 - 0)/(2/3)), \text{abs}((1/4 - 3/4)/(1/3)))$. Concave because the first piece’s slope is greater than the second piece’s.
$\min(\lambda \cdot \text{mult}, 1 - \varepsilon)$.	$\alpha = 1, L = \text{mult}$.	$\text{mult} > 0, \varepsilon > 0$. Piecewise linear; equals 0 at 0, $1 - \varepsilon$ at $(1 - \varepsilon)/\text{mult}$, and $1 - \varepsilon$ at 1. $L = \max(\text{mult}, 0)$. Concave.
$1/10$ if λ is 0; $-1/(2 \cdot \ln(\lambda/2)) + 1/10$ otherwise.	Not Hölder continuous.	Has a slope near 0 that’s steeper than every “nth” root.

Note: A Hölder-continuous function with Hölder exponent α and Hölder constant L is also Hölder continuous with Hölder exponent β and Hölder constant no more than L , where $0 < \beta < \alpha$.

Finding parameters α and L for Hölder-continuous functions. If $f(\lambda)$ is continuous, the following is one way to find the Hölder exponent (α) and Hölder constant (L) of f , to determine whether f is Hölder continuous, not just continuous.

First, if f has a bounded and continuous first derivative on its domain, then α is 1 (f is **Lipschitz continuous**) and L is the maximum of the absolute value of that derivative.

Otherwise, consider the function $h(\lambda, c) = \text{abs}(f(\lambda) - f(c))/((\text{abs}(\lambda - c))^\alpha)$, or 0 if $\lambda = c$, where $0 < \alpha \leq 1$ is a Hölder exponent to test. For a given α , let $g(\lambda)$ be the maximum of $h(\lambda, c)$ over all points c where f has a “vertical slope” or the “steepest slope exhibited”. If $g(\lambda)$ is bounded for a given α on f ’s domain (in this case, the closed unit interval), then f is Hölder continuous with Hölder exponent α and Hölder constant (L) equal to or greater than the maximum value of $g(\lambda)$ on its domain.

The following example, which uses the SymPy computer algebra library, plots $\max(h(\lambda, 0), h(\lambda, 1))$ when $f = \sqrt{\lambda(1-\lambda)}$ and $\alpha = 1/2$: `lamda, c=symbols('lamda c'); func=sqrt(lamda*(1-lamda)); alpha=S(1)/2; h=Abs(func-func.subs(lamda, c))/Abs(lamda-c)**alpha; plot(Max(h.subs(c, 0), h.subs(c, 1)), (lamda, 0, 1))`.

Functions with a Hölder-continuous or Lipschitz-continuous derivative. The following table shows some functions whose derivatives are Hölder continuous, and others where that is not the case. (In the SymPy library, a function’s derivative can be found using the `diff` method.) In the following table, if f has a bounded and continuous *second* derivative on its domain, then α is 1 (the first derivative is Lipschitz continuous) and L is the maximum of the absolute value of that *second* derivative.

Function $f(\lambda)$	Derivative’s Hölder exponent (α) and an upper bound of the derivative’s Hölder constant (L):	Notes
$\lambda^{1+\beta}$	$\alpha = \beta . L = 1 + \beta .$	$0 < \beta \leq 1.$
$3/4 - \sqrt{\lambda(1-\lambda)}.$	Derivative is not Hölder continuous.	Derivative is not Hölder continuous because f is not Lipschitz continuous.
$\cosh(\lambda) - 3/4.$	$\alpha = 1$ (derivative is Lipschitz continuous). $L = \cosh(1).$	Continuous second derivative, namely $\cosh(\lambda)$. Convex. <code>cosh</code> is the hyperbolic cosine function.
$\lambda \cdot \sin(z\lambda)/4 + 1/2.$	$\alpha = 1. L = z(2 + z\lambda)/4.$	Continuous second derivative. L is an upper bound of its absolute value. $z > 0.$
$\sin(z\lambda)/4 + 1/2.$	$\alpha = 1. L = (z^2)/4.$	Continuous second derivative; L is an upper bound of its absolute value, namely $\text{abs}(-\sin(z\lambda) \cdot z^2/4).$ $z > 0.$
$\lambda^2/2 + 1/10$ if $\lambda \leq 1/2$; $\lambda/2 - 1/40$ otherwise.	$\alpha = 1. L = 1.$	Lipschitz-continuous derivative, with Lipschitz constant 1.
$\exp(-\lambda).$	$\alpha = 1. L = 1.$	Lipschitz-continuous derivative, with Lipschitz constant 1. ⁴⁹
$\lambda/2$ if $\lambda \leq 1/2$; $(4^* \lambda - 1)/(8^* \lambda)$ otherwise.	$\alpha = 1. L = 1.$	Concave. Lipschitz-continuous derivative with Lipschitz constant 2.

9.2 Results Used in Approximate Bernoulli Factories

See the appendix of the page “Approximations in Bernstein Form”⁵⁰.

9.3 How Many Coin Flips Are Needed to Simulate a Polynomial?

Let $p(\lambda)$ be a polynomial that maps the closed unit interval to itself and satisfies $0 < p(\lambda) < 1$ whenever $0 < \lambda < 1$.

Then p ’s *coin-flipping degree* (Wästlund 1999)⁵¹ is the smallest value of n such that p ’s Bernstein coefficients of degree n lie in the closed unit interval. ⁵² (This is broader than the use of the term in Wästlund, where a polynomial can have a coin-flipping degree only if its “power” coefficients are integers.) The coin-flipping

⁴⁹This function’s second derivative’s absolute value can be plotted using the SymPy library as follows: `plot(diff(Abs(exp(-x)), (x, 2)), (x, 0, 1))`. In this plot, the maximum is 1, the same as the first derivative’s Lipschitz constant.

⁵⁰<https://peteroupc.github.io/bernapprox.html>

⁵¹Wästlund, J., “**Functions arising by coin flipping**”, 1999. <https://www.math.chalmers.se/~wastlund/coinFlip.pdf>

⁵²The coin-flipping degree is very similar to the so-called *Bernstein degree* or *Lorentz degree*, which is the smallest integer n such that p ’s Bernstein coefficients of degree n are all nonnegative, assuming that p is nonnegative. See, for example, Erdélyi, T., and J. Szabados. “On polynomials with positive coefficients.” *Journal of Approximation Theory* 54.1 (1988): 107-122.

degree is the smallest value of n such that the algorithm of Goyal and Sigman (2012)⁵³ can toss heads with probability $p(\lambda)$ using exactly n biased coin flips (in addition to a fair coin).

The following results give upper bounds on p 's coin-flipping degree.

Suppose p is in Bernstein form of degree m with Bernstein coefficients $b_0, \dots, b_m$:

- If $0 \leq \min(b_0, \dots, b_m) \leq \max(b_0, \dots, b_m) \leq 1$, then the coin-flipping degree is no more than m .
- If $0 \leq \min(b_0, \dots, b_m)$ and $\max(b_0, \dots, b_m) > 1$, then the coin-flipping degree is no more than—

$$\text{iceil} \left(\frac{m(m-1)}{2} \frac{\max(1-b_0, \dots, 1-b_m)}{1 - \text{Pmax}} \right),$$

where $\text{iceil}(x)$ is $x+1$ if x is an integer, or $\text{ceil}(x)$ otherwise, and where Pmax is the maximum value of $p(\lambda)$ on the closed unit interval (Powers and Reznick 2001)⁵⁴; see also Boudaoud et al. (2007)⁵⁵

- If $\min(b_0, \dots, b_m) < 0$ and $\max(b_0, \dots, b_m) \leq 1$, then the coin-flipping degree is no more than—

$$\text{iceil} \left(\frac{m(m-1)}{2} \frac{\max(b_0, \dots, b_m)}{\text{Pmin}} \right),$$

where Pmin is the *minimum* value of $p(\lambda)$ on the closed unit interval (Powers and Reznick 2001)⁵⁶; see also Boudaoud et al. (2007)⁵⁷

- Suppose $m \geq 2$, that $b_0 = 0$ or $b_m = 0$ or both, and that the following necessary conditions are satisfied (Mok and To 2008; Theorem 1 and Corollary 3)⁵⁸:
 - For every i such that $b_i < 0$, if $b_m = 0$, there must be $j > i$ such that $b_j > 0$.
 - For every i such that $b_i < 0$, if $b_0 = 0$, there must be $j < i$ such that $b_j > 0$.
 - For every i such that $b_i > 1$, if $b_m = 1$, there must be $j > i$ such that $b_j < 1$.
 - For every i such that $b_i > 1$, if $b_0 = 1$, there must be $j < i$ such that $b_j < 1$.

Then the coin-flipping degree is no more than—

$$\max(M(b_0, \dots, b_m), M(1-b_0, \dots, 1-b_m)),$$

where—

$$M(\beta_0, \dots, \beta_m) = \text{ceil} \left(\max \left(2m\mu, \frac{m(m-1)\mu^m}{2(1-c)} \frac{a_{\max}}{a_{\min}} \right) \right),$$

and where:

⁵³Vineet Goyal and Karl Sigman. 2012. On simulating a class of Bernstein polynomials. ACM Trans. Model. Comput. Simul. 22, 2, Article 12 (March 2012), 5 pages. <https://doi.org/10.1145/2133390.2133396>

⁵⁴Powers, V., Reznick, B., “A new bound for Pólya’s Theorem with applications to polynomials positive on polyhedra”, *Journal of Pure and Applied Algebra* 164 (24 October 2001). [https://doi.org/10.1016/S0022-4049\(0000155-9](https://doi.org/10.1016/S0022-4049(0000155-9) <https://www.sciencedirect.com/science/article/pii/S0022404900001559>

⁵⁵Boudaoud, F., Caruso, F. & Roy, MF. Certificates of Positivity in the Bernstein Basis. *Discrete Comput Geom* 39, 639–655 (2008). <https://doi.org/10.1007/s00454-007-9042-x>.

⁵⁶Powers, V., Reznick, B., “A new bound for Pólya’s Theorem with applications to polynomials positive on polyhedra”, *Journal of Pure and Applied Algebra* 164 (24 October 2001). [https://doi.org/10.1016/S0022-4049\(0000155-9](https://doi.org/10.1016/S0022-4049(0000155-9) <https://www.sciencedirect.com/science/article/pii/S0022404900001559>

⁵⁷Boudaoud, F., Caruso, F. & Roy, MF. Certificates of Positivity in the Bernstein Basis. *Discrete Comput Geom* 39, 639–655 (2008). <https://doi.org/10.1007/s00454-007-9042-x>.

⁵⁸Mok, H-N., To, W-K., “Effective Pólya semi-positivity for nonnegative polynomials on the simplex”, *Journal of Complexity* 24 (2008). <https://doi.org/10.1016/j.jco.2008.01.003> <https://doi.org/10.1016/j.jco.2008.01.003>

- $a_{\max} = \max(\max(0, \beta_0), \dots, \max(0, \beta_m))$.
- $b_{\max} = \max(-\min(0, \beta_0 \binom{m}{0}), \dots, -\min(0, \beta_m \binom{m}{m}))$.
- $a_{\min}$ is the minimum of $(\beta_i \binom{m}{i})$ over all values of i such that $\beta_i > 0$.
- $\mu = m + Lb_{\max}/a_{\min}$ where L is the number of values of β_i that satisfy $\beta_i < 0$.
- c is the smallest number r that satisfies $FN(\lambda)/FP(\lambda) \leq r$ where $0 < \lambda < 1$. c can also be a greater number but less than 1.
- $FP(\lambda) = \sum_{k=0}^m \max(0, \beta_k) \binom{m}{k} \lambda^k (1-\lambda)^{m-k}$.
- $FN(\lambda) = \sum_{k=0}^m -\min(0, \beta_k) \binom{m}{k} \lambda^k (1-\lambda)^{m-k}$.

(Mok and To 2008; Theorem 2 and remark 1.5(v))⁵⁹.

Additional results are found in Leroy (2008)⁶⁰.

Examples:

1. Let $p(\lambda) = 1 - 8\lambda + 20\lambda^2 - 13\lambda^3$, a polynomial of degree $m = 3$. p 's Bernstein coefficients are $b_0 = 1, b_1 = -5/3, b_2 = 7/3, b_3 = 0$, and its coin-flipping degree is 46 (Wästlund 1999, Example 4.4)⁶¹.
2. An exhaustive search shows that 46 is the highest possible coin-flipping degree for a degree-3 polynomial whose “power” coefficients are integers.
3. The degree-4 polynomial $-43\lambda^4 + 81\lambda^3 - 47\lambda^2 + 9\lambda$ has a coin-flipping degree of 5284.

Note: If a polynomial’s “power” coefficients can be rational numbers (ratios of two integers), even a degree-2 polynomial can have an arbitrarily high coin-flipping degree. An example is the family of degree-2 polynomials $r\lambda - r\lambda^2$, where r is a rational number greater than 0 and less than 4.

Lemma A: Let $a_0, \dots, a_n$ be real numbers, and let $p(\lambda) = a_0\lambda^0 + \dots + a_n\lambda^n$ be a polynomial that maps the closed unit interval to itself. Then $0 \leq a_0 \leq 1$, and the values $a_0, \dots, a_n$ must sum to a value that is 0 or greater and 1 or less.

Proof: This can be seen by evaluating $p(1) = a_0 + \dots + a_n$ and $p(0) = a_0$. If $p(1)$ or $p(0)$ is less than 0 or greater than 1, then p does not meet the hypothesis of the lemma. []

In the following lemmas, let $p(\lambda) = a_0\lambda^0 + \dots + a_n\lambda^n$ be a polynomial that maps the closed unit interval to itself and satisfies $0 < p(\lambda) < 1$ whenever $0 < \lambda < 1$.

Lemma (Chok and Vasil 2025, theorem 3)⁶²: Suppose p is positive on the closed unit interval. Then the Bernstein coefficients of p (once p is written in Bernstein form of degree m) will all be positive as soon as—

$$m > \max \left(n, \text{ceil} \left(\frac{2154n^2 \max(a_0, \dots, a_n)}{\text{Pmin}} \right) \right),$$

where Pmin is the minimum of $p(\lambda)$ on the closed unit interval.

Lemma: If p 's coin-flipping degree is n , then $\text{abs}(a_i) \leq 2^i \binom{n}{i}$.

Proof: Consider the matrix that transforms a polynomial’s Bernstein coefficients to “power” coefficients,

⁵⁹Mok, H-N., To, W-K., “Effective Pólya semi-positivity for nonnegative polynomials on the simplex”, *Journal of Complexity* 24 (2008). <https://doi.org/10.1016/j.jco.2008.01.003> <https://doi.org/10.1016/j.jco.2008.01.003>

⁶⁰Leroy, Richard. “Certificats de positivité et minimisation polynomiale dans la base de Bernstein multivariable.” PhD diss., Université Rennes 1, 2008

⁶¹Wästlund, J., “Functions arising by coin flipping”, 1999. <https://www.math.chalmers.se/~wastlund/coinFlip.pdf>

⁶²Chok, James, and Geoffrey Vasil. “Rational function approximation with normalized positive denominators.” *SIAM Journal on Scientific Computing* 47.5 (2025): A2699-A2721. <https://doi.org/10.1137/24M1632139>

which is $n \times n$ if the polynomial's degree is n (Ray and Nataraj 2012, eq. (8))⁶³. Given the hypothesis of the lemma, each Bernstein coefficient must lie in the closed unit interval and the required matrix size is n , which is p 's coin-flipping degree. For each row of the matrix ($0 \leq i \leq n$), the corresponding “power” coefficient of the polynomial equals a linear combination of that row with a vector of Bernstein coefficients. Thus, the i -th power coefficient equals a_i and its absolute value is no more than $\sum_{m=0}^i \binom{n}{m} \binom{n-m}{i-m} = 2^i \binom{n}{i}$. []

Lemma: $\text{abs}(a_i) \leq \text{abs}(b_i)$, where b_i is the corresponding power coefficient of the following polynomial:

$$q_n(\lambda) = b_0\lambda^0 + \dots + b_n\lambda^n = (T_n(1 - 2\lambda) + 1)/2,$$

and where $T_n(x)$ is the **Chebyshev polynomial of the first kind**⁶⁴ of degree n .⁶⁵

Proof: For any q_n , the coefficient b_0 is 1. Combined with Lemma A, the proof is done for that coefficient. Now for the remaining coefficients $a_1, \dots, a_n$ and $b_1, \dots, b_n$. Given that adding 1 to q_n doesn't change these coefficients, simplify q_n to $(T_n(1 - 2\lambda))/2$. Given that $T_n(1 - 2\lambda)/2 = (-1)^n T_n(2\lambda - 1)/2$, these coefficients have the same absolute value on both sides, so rewrite q_n again to $(T_n(2\lambda - 1))/2$. Finally, even after doubling $a_1, \dots, a_n$ and $b_1, \dots, b_n$ they still remain comparable, so rewrite q_n to $(T_n(2\lambda - 1))$. Then the result follows from lemma 4 of Dubickas (2021)⁶⁶.

Lemma: When p is written in Bernstein form with coefficients $\beta_1, \beta_2, \dots, \beta_n$, then $\text{abs}(2\beta_i - 1) \leq \text{abs}(d_i) = \binom{2n}{2i} / \binom{n}{i}$, where d_i is the corresponding Bernstein coefficient of the polynomial $T_n(2\lambda - 1)$.

The proof follows from Theorem 3 of Lubinsky and Ziegler (1990)⁶⁷ and the proof of Theorem 3.3 of Popa and Raşa (2013)⁶⁸. As a result, the polynomials $q_n(\lambda)$ exhibit the highest possible *offshoot* for a p within the lemma's scope. The *offshoot* is the farthest away a polynomial's Bernstein coefficients $(\beta_0, \dots, \beta_n)$ are from the closed unit interval, calculated as $\max(\max(-\beta_0, \beta_0 - 1, 0), \dots, \max(-\beta_n, \beta_n - 1, 0))$. (For example, q_3 has Bernstein coefficients $[1, -2, 3, 0]$ and an offshoot of 2, because the coefficient 3 is 2 away from the closed unit interval.) The following table shows the offshoot for degrees from 2 through 12:

Degree	Offshoot
2	1
3	2
4	$16/3 < 5.334$
5	10
6	$113/5 = 22.6$
7	$212/5 = 42.4$
8	$640/7 < 94.429$
9	$1212/7 < 173.143$
10	$23063/63 < 366.08$
11	$2098/3 < 699.334$

⁶³S. Ray, P.S.V. Nataraj, “**A Matrix Method for Efficient Computation of Bernstein Coefficients**”, Reliable Computing 17(1), 2012. <https://interval.louisiana.edu/reliable-computing-journal/volume-17/reliable-computing-17-pp-40-71.pdf>

⁶⁴<https://mathworld.wolfram.com/ChebyshevPolynomialoftheFirstKind.html>

⁶⁵This statement was slightly modified from a statement with a proof by Fedor Petrov on *MathOverflow*. <https://mathoverflow.net/questions/449135>

⁶⁶Dubickas, Artūras. “Inequalities between height and deviation of polynomials” Open Mathematics, vol. 19, no. 1, 2021, pp. 540-550. <https://doi.org/10.1515/math-2021-0055>

⁶⁷Lubinsky, D. S., and Z. Ziegler. “Coefficient bounds in the Lorentz representation of a polynomial”. Canadian Mathematical Bulletin 33.2 (1990): 197-206. <https://doi.org/10.4153/CMB-1990-033-1> <https://www.cambridge.org/core/journals/canadian-mathematical-bulletin/article/coefficient-bounds-in-the-lorentz-representation-of-a-polynomial/DF34F2E23B329A44C3B37E8C11073AE4>

⁶⁸Dorian Popa, Ioan Raşa, “On the stability of some classical operators from approximation theory”, *Expositiones Mathematicae* 31(3), 2013, <https://doi.org/10.1016/j.exmath.2013.01.007> <https://www.sciencedirect.com/science/article/pii/S072308691300008X>

Degree	Offshoot
12	48272/33 < 1462.788

It seems that this offshoot approaches $c \cdot 2^n$ as n increases, where c is a number that approaches about 0.353. Stated differently, the offshoot seems to grow like $O(2^n)$, at an exponential rate in the degree of the polynomial.

9.4 Proofs for Polynomial-Building Schemes

This section shows mathematical proofs for some of the polynomial-building schemes of this page.

In the following results:

- A *modulus of continuity* of a function f , denoted $\omega(\delta)$, means a nonnegative and nowhere decreasing function where, for each $\delta \geq 0$, $\text{abs}(f(x) - f(y)) \leq \omega(\delta)$ whenever x and y are in f 's domain and no more than δ apart. Loosely speaking, $\omega(\delta)$ gives how much f can vary when f is restricted to a window of size δ or less. The modulus of continuity reflects the “regularity” of f ; generally, the smaller it is, the more “regular”.

Some of the results rely on properties of random variables. For a random variable Y , define—

- $\mathbb{E}[Y]$ as the *mean* (or expected value or “long-run average”) of Y , and
- $\tau_r(Y)$ as the r -th *central absolute moment* of Y , that is, $\tau_r(Y) = \mathbb{E}[\text{abs}(Y - \mathbb{E}[Y])^r]$.

Lemma 6(i) of Nacu and Peres (2005)⁶⁹ finds a bound involving hypergeometric random variables that is essential in polynomial-building schemes. This part of the lemma can be applied to continuous functions beyond just Lipschitz-continuous functions. This includes the larger class of *Hölder-continuous* functions (see “**Definitions**”).

Lemma 2. *Let $f(\lambda)$ be a continuous function that maps the closed unit interval to itself, let X be a hypergeometric($2^*n, k, n$) random variable, and let n be a positive integer.*

1. *Let $\omega(\delta)$ be a modulus of continuity of f . If ω is continuous and concave, then the expression—*

$$\text{abs}(\mathbb{E}[f(X/n)] - f(k/(2n))), \quad (1)$$

is less than or equal to each of the following:

- $\omega(\sqrt{1/(8^*n - 4)})$.
 - $\omega(\sqrt{1/(7^*n)})$ if $n \geq 4$.
 - $\omega(\sqrt{1/(2^*n)})$.
 - $\omega(\sqrt{(k/(2^*n)) * (1 - k/(2^*n)) / (2^*n - 1)})$.
2. *If f is Hölder continuous with Hölder constant M or less and with Hölder exponent α such that $0 < \alpha \leq 1$, then the expression (1) is less than or equal to—*
 - $M^*(1/(2^*n))^{\alpha/2}$,
 - $M^*(1/(7^*n))^{\alpha/2}$ if $n \geq 4$, and
 - $M^*(1/(8^*n - 4))^{\alpha/2}$.
 3. *If f has a derivative that is Hölder continuous with Hölder constant M or less and with Hölder exponent α , then the expression (1) is less than or equal to—*
 - $(M/2)^*(1/(7^*n))^{(1+\alpha)/2}$ if $n \geq 4$, and

⁶⁹Nacu, Șerban, and Yuval Peres. “Fast simulation of new coins from old”, The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

- $(M/2)^*(1/(8^n - 4))^{(1+\alpha)/2}$.

Proof. Note that $\mathbb{E}[X/n] = k/(2n)$, and $\tau_2[X/n] \leq (k/(2n)) \cdot (1 - k/(2n))/(2n - 1) \leq 1/(8n - 4)$. Through Lemma 5 of “**Notes on Approximation Theory**”⁷⁰, there is a positive linear operator $L(f)(\lambda) = \mathbb{E}[f(X/n)]$ with $\lambda = \mathbb{E}[X/n] = k/(2n)$. Parts 1 and 2 then follow from inequalities 2 and 3 of Lemma 2 of that article, using more or less tight bounds for $\tau_2[X/n]$. Part 3 follows from inequality 6 of the same lemma, plugging in estimates for the variance of X/n found in part 1. []

Notes:

1. A hypergeometric($2^n, k, n$) random variable is the number of “good” balls out of k balls taken uniformly at random, all at once, from a bag containing 2^n balls, n of which are “good”.
2. Parts 1 through 3 exploit a tighter bound on the variance of X/n than the bound given in Nacu and Peres (2005, Lemma 6(i) and 6(ii), respectively)⁷¹.
3. All continuous functions that map the closed unit interval to itself, including all of them that admit a Bernoulli factory, have a modulus of continuity. The proof of part 1 remains valid even if $\omega(0) > 0$, because the bounds proved remain correct even if ω is overestimated. Certain functions have a simple modulus of continuity; for details, see Remark 2 of “**Notes on Approximation Theory**”⁷².

Theorem 1. *Let f be continuous on the closed unit interval and have a minimum greater than 0 and a maximum less than 1, let n_0 be a positive integer, and let $\phi(n)$ be a function that takes on a nonnegative value. Suppose f is such that the expression (1) in Lemma 2 is less than or equal to $\phi(n)$ whenever $n \geq n_0$ is an integer power of 2. Let—*

$$\eta(n) = \sum_{k \geq \log_2(n)} \phi(2^k),$$

for every integer $n \geq 1$ that’s a power of 2. If the series $\eta(n)$ converges to a finite value for each such n , and if it converges to 0 as n gets large, then the following scheme for $f(\lambda)$ is valid in the following sense:

- There are polynomials g_n and h_n (where n is a positive integer power of two) as follows. The k -th Bernstein coefficient of g_n and h_n is **fbelow**(n, k) and **fabove**(n, k), respectively (where $0 \leq k \leq n$), where:
 - If $n_0 = 1$: **fbelow**(n, k) = $f(k/n) - \eta(n)$; **fabove**(n, k) = $f(k/n) + \eta(n)$.
 - If $n_0 > 1$:
 - * **fbelow**(n, k) = $\min(\text{fbelow}(n_0, 0), \text{fbelow}(n_0, 1), \dots, \text{fbelow}(n_0, n_0))$ if $n < n_0$; $f(k/n) - \eta(n)$ otherwise.
 - * **fabove**(n, k) = $\max(\text{fabove}(n_0, 0), \text{fabove}(n_0, 1), \dots, \text{fbelow}(n_0, n_0))$ if $n < n_0$; $f(k/n) + \eta(n)$ otherwise.

If the scheme is valid in the sense just given, then there are also polynomials g_n and h_n for each integer $n > 1$ other than a power of 2, whose Bernstein coefficients are **fbelow** and **fabove** as before, where:

- **fbelow**(n, k) = $(k/n) * \text{fbelow}(n - 1, \max(0, k - 1)) + ((n - k)/n) * \text{fbelow}(n - 1, \min(n - 1, k))$, and
- **fabove**(n, k) = $(k/n) * \text{fabove}(n - 1, \max(0, k - 1)) + ((n - k)/n) * \text{fabove}(n - 1, \min(n - 1, k))$,

The polynomials g_n and h_n satisfy:

1. $g_n \leq h_n$.

⁷⁰<https://peteroupc.github.io/approxtheory.html>

⁷¹Nacu, Șerban, and Yuval Peres. “Fast simulation of new coins from old”, The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

⁷²<https://peteroupc.github.io/approxtheory.html>

2. g_n and h_n converge to f as n gets large.
3. $(g_{n+1} - g_n)$ and $(h_n - h_{n+1})$ are polynomials with nonnegative Bernstein coefficients once they are rewritten to polynomials in Bernstein form of degree exactly $n + 1$.

Proof. For simplicity, this proof assumes first that $n_0 = 1$. First, deal with the case that n is a power of 2.

For the series $\eta(n)$ in the theorem, because $\phi(n)$ is nonnegative, each term of the series is nonnegative making the series nonnegative and, by the assumption that the series converges, $\eta(n)$ is nowhere increasing with increasing n .

Item 1 is trivial. If $n \geq n_0$, g_n is simply the Bernstein polynomial of f minus a nonnegative value, and h_n is the Bernstein polynomial of f plus that same value, and if n is less than n_0 , g_n is a constant value not less than the lowest point reachable by the lower polynomials, and h_n is a constant value not less than the highest point reachable by the upper polynomials.

Item 2 is likewise trivial. A well known result is that the Bernstein polynomials of f converge to f as their degree n gets large. And because the series η (in Theorem 1) sums to a finite value that goes to 0 as n increases, the upper and lower shifts will converge to 0 so that g_n and h_n converge to the degree- n Bernstein polynomials and thus to f .

Item 3 is the *consistency requirement* described earlier in this page. This is ensured as in Proposition 10 of Nacu and Peres (2005)⁷³ by bounding, from below, the offset by which to shift the approximating polynomials. This lower bound is $\eta(n)$, a solution to the equation $0 = \eta(n) - \eta(2 * n) - \phi(n)$ (see note later), where $\phi(n)$ is a function that takes on a nonnegative value.

$\phi(n)$ is, roughly speaking, the minimum distance between one polynomial and the next so that the consistency requirement is met between those two polynomials. Compare the assumptions on ϕ in Theorem 1 with equations (10) and (11) in Nacu and Peres (2005).

The solution for $\eta(n)$ given in the statement of the theorem is easy to prove by noting that this is a recursive process: we start by calculating the series for $n = 2 * n$, then adding $\phi(n)$ to it (which will be positive), in effect working backwards and recursively, and we can easily see that we can calculate the series for $n = 2 * n$ only if the series converges, hence the assumption of a converging series.

Now to prove the result assuming that $n_0 > 1$, again dealing with the case that n is a power of 2.

Doing this involves taking advantage of the observation in Remark B of Nacu and Peres (2005)⁷⁴ that we can start defining the polynomials at any n greater than 0, including $n = n_0$; in that case, the upper and lower polynomials of degree 1 or greater, but less than n_0 , would be constant functions, so that the Bernstein coefficients of each polynomial would be equal. The lower constants are no greater than g_{n_0} 's lowest Bernstein coefficient, and the upper constants are no less than g_{n_0} 's highest Bernstein coefficients; they meet Item 3 because these lower and upper constants, when rewritten as polynomials in Bernstein form of degree n_0 , have Bernstein coefficients that are still no greater or no less, respectively, than the corresponding degree- n_0 polynomial. With the ϕ given in this theorem, the series $\eta(n)$ in the theorem remains nonnegative. Moreover, since η is assumed to converge, $\eta(n)$ still decreases with increasing n .

For the case where n is a power of 2, the result also follows from Remark B of Nacu and Peres (2005)⁷⁵. []

Notes:

1. There is only one solution $\eta(n)$ in the case at hand. Unlike so-called **functional equations**⁷⁶ and linear recurrences, with a solution that varies depending on the starting value,

⁷³Nacu, Șerban, and Yuval Peres. "Fast simulation of new coins from old", The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

⁷⁴Nacu, Șerban, and Yuval Peres. "Fast simulation of new coins from old", The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

⁷⁵Nacu, Șerban, and Yuval Peres. "Fast simulation of new coins from old", The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

⁷⁶<https://math.stackexchange.com/questions/3993739>

there is only one solution in the case at hand, namely the solution that makes the series converge, if it exists at all. Alternatively, the equation can be expanded to $0 = \eta(n) - \eta(4 * n) - \phi(2 * n) - \phi(n) = \eta(n) - \eta(8 * n) - \phi(4 * n) - \phi(2 * n) - \phi(n) = \dots$

2. $\log_2(n)$ is the number x such that $2^x = n$.

Proposition 1A. *If a scheme satisfies Theorem 1, then given only polynomials g_n and h_n in the scheme where n is a power of two, the polynomials can be made to satisfy conditions (i), (iii), and (iv) of Proposition 3 of Nacu and Peres (2005)⁷⁷ as follows:*

- $g_n = g_{n-1}$ and $h_n = h_{n-1}$ whenever n is an integer greater than 1 and not a power of 2.
- If **fabove**(n, k) > 1 for a given n and some k , the Bernstein coefficients of h_n (the upper polynomial) are all 1.
- If **fbelow**(n, k) < 0 for a given n and some k , the Bernstein coefficients of g_n (the lower polynomial) are all 0.

Proof: Condition (i) of Proposition 3 says that each Bernstein coefficient of the polynomials must be 0 or greater and 1 or less. This is ensured starting with a large enough value of n greater than 0 that's a power of 2, call it n_1 , as shown next.

Let ε be a positive distance between 0 and the minimum or between 1 and the maximum of f , whichever is smaller. This ε exists by the assumption that f is bounded away from 0 and 1. Because the series η (in Theorem 1) sums to a finite value that goes to 0 as n increases, $\eta(n)$ will eventually stay less than ε . And if $n \geq n_0$ is a power of 2 (where n_0 is as in Theorem 1), the $\mathbf{f}(\mathbf{k}/\mathbf{n})$ term is bounded by the minimum and maximum of f by construction. This combined means that the lower and upper polynomials' Bernstein coefficients will eventually be bounded by 0 and 1 for every integer n starting with n_1 .

For n less than n_1 , condition (i) is ensured by setting the lower or upper polynomial's Bernstein coefficient to 0 or 1, respectively, whenever a Bernstein coefficient of the degree- n polynomial would otherwise be less than 0 or greater than 1, respectively.

Condition (iii) of Proposition 3 is mostly ensured by item 2 of Theorem 1. The only thing to add is that for n less than n_1 , the lower and upper polynomials g_n and h_n can be treated as 0 or 1, respectively, without affecting convergence, and that for n other than a power of 2, defining $g_n = g_{n-1}$ and $h_n = h_{n-1}$ maintains condition (iii) by Remark B of Nacu and Peres (2005)⁷⁸.

Condition (iv) of Proposition 3 is mostly ensured by item 3 of Theorem 1. For $n=n_1$, condition (iv) is maintained by noting that the degree- n_1 polynomial's Bernstein coefficients must be bounded by 0 and 1 by condition (i) so they will likewise be bounded by those of the lower and upper polynomials of degree less than n_1 , and those polynomials are the constant 0 and the constant 1, respectively, as are their Bernstein coefficients. Finally, for n other than a power of 2, defining $g_n = g_{n-1}$ and $h_n = h_{n-1}$ maintains condition (iv) by Remark B of Nacu and Peres (2005)⁷⁹. []

Note: The last condition of Proposition 3, condition (ii), says **fabove**(n, k)*choose(n, k) and **fbelow**(n, k)*choose(n, k) must be integers.⁸⁰ But Proposition 3 assumes only the biased coin and no other randomness is used, and that the coin doesn't show heads every time or tails every time. Therefore, $f(0)$, if it exists, must be an integer, and the same is true for $f(1)$, so that

⁷⁷Nacu, Șerban, and Yuval Peres. "Fast simulation of new coins from old", The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

⁷⁸Nacu, Șerban, and Yuval Peres. "Fast simulation of new coins from old", The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

⁷⁹Nacu, Șerban, and Yuval Peres. "Fast simulation of new coins from old", The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

⁸⁰choose(n, k) = $(1*2*3*...*n)/((1*...*k)*(1*...(n-k))) = n!/(k! * (n-k)!) = \binom{n}{k}$ is a *binomial coefficient*, or the number of ways to choose k out of n labeled items. It can be calculated, for example, by calculating $i/(n-i+1)$ for each integer i satisfying $n-k+1 \leq i \leq n$, then multiplying the results (Yannis Manolopoulos. 2002. "Binomial coefficient computation: recursion or iteration?", SIGCSE Bull. 34, 4 (December 2002), 65-67. DOI: <https://doi.org/10.1145/820127.820168>). For every $m > 0$, choose($m, 0$) = choose(m, m) = 1 and choose($m, 1$) = choose($m, m-1$) = m ; also, in this document, choose(n, k) is 0 when k is less than 0 or greater than n . $n!$ = $1*2*3*...*n$ is also known as n factorial; in this document, $(0!) = 1$.

condition (ii) is redundant with condition (iii) due to a result that goes back to Kantorovich (1931)⁸¹; see also Remark C of Nacu and Peres (2005)⁸².

Corollary 1. *Let $f(\lambda)$ be continuous on the closed unit interval and have a minimum greater than 0 and a maximum less than 1. If that function is Hölder continuous with Hölder constant M or less and Hölder exponent α , then each of the following schemes is valid in the sense of Theorem 1:*

- $n_0 = 1$ and $\eta(n) = M/((2^{\alpha/2} - 1)n^{\alpha/2})$.
- $n_0 = 4$ and $\eta(n) = M(2/7)^{\alpha-2}/((2^{\alpha/2} - 1)n^{\alpha/2})$.

If f is Lipschitz continuous with Lipschitz constant M or less, then each of the following schemes is valid in the sense of Theorem 1:

- $n_0 = 1$ and $\eta(n) = M/((\sqrt{2} - 1)\sqrt{n})$.
- $n_0 = 4$ and $\eta(n) = M\sqrt{2/7}/((\sqrt{2} - 1)\sqrt{n})$.

Proof. Because f is Hölder continuous, it admits the modulus of continuity $\omega(x) = Mx^\alpha$. By part 1 of lemma 2:

- For each integer $n \geq 1$ that's a power of 2 ($n_0 = 1$ in Theorem 1), $\phi(n) = \omega(\sqrt{1/(2n)}) = M(1/(2n))^{\alpha/2}$ can be taken for each such integer n , and thus $\eta(n) = D(n) = \frac{M}{((2^{\alpha/2}-1)n^{\alpha/2})}$ (where $\eta(n)$ is as in Theorem 1).
- For each integer $n \geq 4$ that's a power of 2 ($n_0 = 4$ in Theorem 1), $\phi(n) = \omega(\sqrt{1/(2n)}) = M(1/(7n))^{\alpha/2}$ can be taken for each such integer n , and thus $\eta(n) = M^*(2/7)^{\alpha/2}/((2^{\alpha/2} - 1)n^{\alpha/2})$.

In both cases $\eta(n)$ is finite and converges to 0 as n increases.

The first and second schemes then follow from Theorem 1. The third and fourth schemes follow from the first and second because Lipschitz-continuous functions with Lipschitz constant M are Hölder continuous with Hölder constant M and exponent 1. []

Notes:

1. For specific values of α , the equation $D(n) = D(2 * n) + \phi(n)$ can be solved via linear recurrences; an example for $\alpha = 1/2$ is the following code in Python that uses the SymPy computer algebra library: `alpha=(S(1)/2); rsolve(Eq(f(n), f(n+1)+z*(1/(2*2**n))**(alpha/2)), f(n)).subs(n,ln(n,2)).simplify()`. Trying different values of α suggested the following formula for Hölder-continuous functions with α of $1/j$ or greater: $(M^* \sum_{i=0}^{2j-1} 2^{i/(2j)})/n^{1/(2)} = M / ((2^{1/(2)} - 1)n^{1/(2)})$; and generalizing the latter expression led to the term in the theorem.
2. The third scheme given here is a special case of Theorem 1 that was already found by Nacu and Peres (2005)⁸³.

Corollary 3. *Let $f(\lambda)$ be continuous on the closed unit interval and have a minimum greater than 0 and a maximum less than 1. If that function has a Lipschitz-continuous derivative with Lipschitz constant L , then a scheme where $n_0 = 4$ and $\eta(n) = L/(7n)$ is valid in the sense of Theorem 1.*

Proof. By part 3 of lemma 2, for each integer $n \geq 4$ that's a power of 2 ($n_0 = 4$ in Theorem 1), $\phi(n) = (L/2)(1/(7n))$ can be taken for each such integer n , and thus $\eta(n) = L/(7n)$ (where $\eta(n)$ is as in Theorem 1). $\eta(n)$ is finite and converges to 0 as n increases. The result then follows from Theorem 1. []

⁸¹Kantorovich, L.V., "Some remarks on the approximation of functions by means of polynomials with integer coefficients", 1931.

⁸²Nacu, Șerban, and Yuval Peres. "Fast simulation of new coins from old", The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

⁸³Nacu, Șerban, and Yuval Peres. "Fast simulation of new coins from old", The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

Note: Nacu and Peres (2005)⁸⁴ already proved a looser scheme in the case when f has a second derivative on the closed unit interval whose maximum absolute value is finite (a slightly stronger condition than having a Lipschitz-continuous derivative on that domain).

Theorem 2. —Let $f(\lambda)$ be continuous on the closed unit interval and have a minimum greater than 0 and a maximum less than 1. If that function is convex and nowhere decreasing, then Theorem 1 remains valid with $\phi(n) = \mathbb{E}[f(Y/n)]$ (where Y is a hypergeometric($2^*n, n, n$) random variable), rather than as given in that theorem. —

Proof. Follows from Theorem 1 and part 4 of Lemma 2 above. With the ϕ given in this theorem, the series $\eta(n)$ in Theorem 1 remains nonnegative; also, this theorem adopts Theorem 1’s assumption that the series converges, so that $\eta(n)$ still decreases with increasing n . []

Proposition 1.

1. Let f be as given in Theorem 1 or 2 or Corollary 1 or 3, except that f must be concave and may have a minimum of 0. Then the schemes of those results remain valid if **fbelow**(n, k) = $f(k/n)$, rather than as given in those results.
2. Let f be as given in Theorem 1 or 2 or Corollary 1 or 3, except that f must be convex and may have a maximum of 1. Then the schemes of those results remain valid if **fabove**(n, k) = $f(k/n)$, rather than as given in those results.

Proof. Parts 1 and 2 follow from Theorem 1 or 2 or Corollary 1 or 3, as the case may be. For part 1, the lower polynomials are replaced by the degree- n Bernstein polynomials of f , and they meet the conditions in those theorems by Jensen’s inequality. For part 2, the upper polynomials are involved instead of the lower polynomials. []

The following lemma shows that if a scheme for $f(\lambda)$ shifts polynomials upward and downward, the pre-shifted polynomials are close to $f(\lambda)$ by the amount of the shift.

In the following lemma, a polynomial $P(x)$ is written in *Bernstein form of degree n* if it is written as—

$$P(x) = \sum_{k=0}^n a_k \binom{n}{k} x^k (1-x)^{n-k},$$

where the real numbers $a_0, \dots, a_n$ are the polynomial’s *Bernstein coefficients*. The degree- n *Bernstein polynomial* of an arbitrary function $f(x)$ is denoted $B_n(f)$ and has Bernstein coefficients $a_k = f(k/n)$. In general, this Bernstein polynomial differs from f even if f is a polynomial.

Lemma 3. Let f be continuous on the closed unit interval and have a minimum greater than 0 and a maximum less than 1. Let S be an infinite set of positive integers. For each integer $n \geq 1$, let $W_n(\lambda)$ be a function, and let $\epsilon_n(f)$ be a nonnegative constant that depends on f and n . Suppose that there are polynomials g_n and h_n (for each n in S) as follows:

1. g_n and h_n have Bernstein coefficients $W_n(k/n) - \epsilon_n(f)$ and $W_n(k/n) + \epsilon_n(f)$, respectively ($0 \leq k \leq n$).
2. $g_n \leq h_n$.
3. g_n and h_n converge to f as n gets large.
4. $(g_m - g_n)$ and $(h_n - h_m)$ are polynomials with nonnegative Bernstein coefficients once they are rewritten to polynomials in Bernstein form of degree exactly m , where m is the smallest number greater than n in S .

Then for each n in S , $\text{abs}(f(\lambda) - B_n(W_n(\lambda))) \leq \epsilon_n(f)$ whenever $0 \leq \lambda \leq 1$.

Proof: $W_n(k/n)$ is the k -th Bernstein coefficient of $B_n(W_n(\lambda))$, which is g_n and h_n before they are shifted downward and upward, respectively, by $\epsilon_n(f)$. Moreover, property 4 in the lemma corresponds to condition

⁸⁴Nacu, Şerban, and Yuval Peres. “Fast simulation of new coins from old”, The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

(iv) of Nacu and Peres (2005)⁸⁵, which implies that, for every $m > n$, $g_n(\lambda) \leq g_m(\lambda) \leq f(\lambda)$ (the lower polynomials “increase”) and $h_n(\lambda) \geq h_m(\lambda) \geq f(\lambda)$ (the upper polynomials “decrease”) for every $n \geq 1$ (Nacu and Peres 2005, Remark A)⁸⁶.

Then if $B_n(W_n(\lambda)) < f(\lambda)$ for some λ in the closed unit interval, shifting the left-hand side upward by $\epsilon_n(f)$ (a nonnegative constant) means that $h_n = B_n(W_n(\lambda)) + \epsilon_n(f) \geq f(\lambda)$, and rearranging this expression leads to $f(\lambda) - B_n(W_n(\lambda)) \leq \epsilon_n(f)$.

Likewise, if $B_n(W_n(\lambda)) > f(\lambda)$ for some λ in the closed unit interval, shifting the left-hand side downward by $\epsilon_n(f)$ means that $g_n = B_n(W_n(\lambda)) - \epsilon_n(f) \leq f(\lambda)$, and rearranging this expression leads to $B_n(W_n(\lambda)) - f(\lambda) \leq \epsilon_n(f)$.

This combined means that $\text{abs}(f(x) - B_n(W_n(\lambda))) \leq \epsilon_n(f)$ whenever $0 \leq \lambda \leq 1$. []

Corollary 4. *If $f(\lambda)$ satisfies a scheme given in Theorem 1 with $n_0 \geq 1$, then $B_n(f(\lambda))$ comes within $\eta(n)$ of f for every integer $n \geq n_0$ that's a power of 2; that is, $\text{abs}(B_n(f(\lambda))) \leq \eta(n)$ for every such n .*

Lemma 5. *Let n be a positive integer. Suppose g_{2n} and g_n are polynomials in Bernstein form of degree $2n$ and n , respectively, and their domain is the closed unit interval. Suppose g_{2n} and g_n satisfy the following property: $(g_{2n} - g_n)$ is a polynomial with nonnegative Bernstein coefficients once it is rewritten to a polynomial in Bernstein form of degree exactly $2n$. Then for every $x \geq 0$, $g_{2n} + x$ and g_n satisfy that property, and for every $x \geq 1$, $g_{2n} \cdot x$ and g_n do as well.*

The proof follows from two well-known properties of polynomials in Bernstein form: adding x to g_{2n} amounts to adding x to its Bernstein coefficients, and multiplying g_{2n} by x amounts to multiplying its Bernstein coefficients by x . In either case, if x is nonnegative, g_{2n} 's Bernstein coefficients become no less than they otherwise would, so that $(g_{2n} - g_n)$ continues to have nonnegative Bernstein coefficients as required by the property.

It is also true that $g_{2n} \cdot x$ and $g_n \cdot x$ satisfy the same property for every $x \geq 0$, and $g_{2n} + y$ and $g_n + y$ do so for every real number y , but a detailed proof of this is left as an exercise to anyone interested. (If $x = 0$, $g_n \cdot x = g_{2n} \cdot x = 0$, $g_n + x = g_n$, and $g_{2n} + x = g_{2n}$, so that the property is trivially satisfied.)

9.4.1 A Conjecture on Polynomial Approximation

The following conjecture suggests there may be a way to easily adapt other approximating polynomials, besides the ordinary Bernstein polynomials, to the Bernoulli factory problem.

Conjecture.

Let r be a positive integer, and let $f(\lambda)$ be a function that—

- is defined on the closed unit interval (that is, $0 \leq \lambda \leq 1$),
- has a minimum of greater than 0 and a maximum of less than 1, and
- has a continuous r -th derivative.

Let M be the maximum of the absolute value of f and its derivatives up to the r -th derivative. Let $W_{2^0}(\lambda), W_{2^1}(\lambda), \dots, W_{2^n}(\lambda), \dots$ be bounded functions on the closed unit interval that converge uniformly to f (that is, for every tolerance level, all W_{2^i} after some value i are within that tolerance level of f at all points on the closed unit interval).

Suppose that there is $D > 0$ such that, for each integer $n \geq 1$ that's a power of 2—

$$\text{abs}(f(\lambda) - B_n(W_n(\lambda))) \leq DM/n^{r/2},$$

⁸⁵Nacu, Şerban, and Yuval Peres. “Fast simulation of new coins from old”, The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

⁸⁶Nacu, Şerban, and Yuval Peres. “Fast simulation of new coins from old”, The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

whenever $0 \leq \lambda \leq 1$, where $B_n(W_n(\lambda))$ is the degree- n Bernstein polynomial of $W_n(\lambda)$.

Then there is a number $C_0 \geq D$ such that the polynomials (g_n) in Bernstein form of degree $2, 4, 8, \dots, 2^i, \dots$, defined as $g_n = B_n(W_n(\lambda)) - C_0 M / n^{r/2}$, converge from below to f with increasing n and have the following property: For each n that's a power of two, $(g_{2n} - g_n)$ is a polynomial with nonnegative Bernstein coefficients once it's rewritten to a polynomial in Bernstein form of degree exactly $2n$.

Equivalently (see also Nacu and Peres 2005), there is $C_1 > 0$ such that the inequality—

$$W_{2n} \left(\frac{k}{2n} \right) + \frac{C_1 M}{n^{r/2}} - \sum_{i=0}^n W_n \left(\frac{i}{n} \right) \sigma_{n,k,i} \geq 0, \quad (\text{PB})$$

$$\text{or } \sum_{i=0}^n \left(W_{2n} \left(\frac{k}{2n} \right) - W_n \left(\frac{i}{n} \right) \right) \sigma_{n,k,i} \geq -\frac{C_1 M}{n^{r/2}},$$

holds true for each integer $n \geq 1$ that's a power of 2 and whenever $0 \leq k \leq 2n$. Here, $\sigma_{n,k,i} = \binom{n}{i} \binom{n}{k-i} / \binom{2n}{k}$ is the probability that a hypergeometric($2^*n, k, n$) random variable equals i .

It is further conjectured that the same value of C_0 (or C_1) suffices when f has a Lipschitz-continuous $(r-1)$ -th derivative and M is the maximum of the absolute value of f and the Lipschitz constants of f and its derivatives up to the $(r-1)$ -th derivative.

C_0 or C_1 may depend on r and the sequence W_n , but not on f , λ , or n . When C_0 or C_1 exists, find a good upper bound for it.

Notes:

1. By Lemma 3, $B_n(W_n(f(\lambda)))$ would be close to $f(\lambda)$ by at most $C_0 M / n^{r/2}$. Properties 2 and 3 in the conjecture correspond to (iii) and (iv) in Nacu and Peres (2005, Proposition 3)⁸⁷.
2. This conjecture is true in particular cases. For example, it is true if W_n converges uniformly to f and, for each n and whenever $\lambda = i/n$ where $0 \leq i \leq n$:
 - $W_n(\lambda) = c(n)$, where $c(n)$ is a number that depends only on n (in that case, $C_0 = D$), or
 - $W_n(\lambda) = f(\lambda)$ and r is 1 or 2 (Nacu and Peres 2005)⁸⁸.⁸⁹
3. For any C_1 that satisfies the conjecture, C_0 can be taken as $C_0 = \max(D, C_1 / (1 - 1/2^{r/2}))$. See Theorem 1.

The following results on hypergeometric random variables are useful. Denote the s -th central absolute moment of a random variable Y as $\tau_s(Y)$.

⁸⁷Nacu, Şerban, and Yuval Peres. “**Fast simulation of new coins from old**”, The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

⁸⁸Nacu, Şerban, and Yuval Peres. “**Fast simulation of new coins from old**”, The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

⁸⁹When r is 1 or 2, this case includes cases where W_n equals f at $0, 1/n, \dots, n/n$ and converges uniformly to f . One example is where W_n is the Lagrange interpolating polynomial of degree n ; given that r is 1 or 2, f is continuous and of bounded variation, so that W_n converges uniformly to f by a result of P. Vértési, “**Lagrange interpolation for continuous functions of bounded variation**”, *Acta Mathematica Hungarica* 35(1), 1980. Another example is where W_n is a rational interpolation described in Zhang and Liu, “**Rational interpolation operator with finite Lebesgue constant**”, *Calcolo* 59.1 (2022). <https://doi.org/10.1007/BF01896819> <https://doi.org/10.1007/s10092-021-00454-1>

Lemma C1.⁹⁰ Let X be a hypergeometric($2n, k, n$) random variable, where n and k are integers satisfying $n \geq 1$ and $0 \leq k \leq 2n$. Then, for every even integer $s \geq 0$:

$$\tau_s(X) \leq \frac{s!}{8^{s/2}((s/2)!)} n^{s/2} = \sigma(s, n).$$

For every odd integer $s \geq 1$:

$$\tau_s(X) \leq \sqrt{\tau_{s-1}(X)\tau_{s+1}(X)} \leq \sqrt{\sigma(s-1, n)\sigma(s+1, n)}.$$

Proof: If $k > n$, rewrite X to a hypergeometric($2n, n, k$) random variable, which has the same distribution as the original, and set $k = n$, since the sample size is now n . (See also Johnson et al. (2005)⁹¹.) Now it can be assumed without loss of generality that $0 > n > k$.

It is known that, if s is an even integer, the s -th central moment of any real-number random variable, including X , is equal to the s -th central absolute moment.

The s -th central moment of X is the expected value or “long-run average” of a function of X , namely $(X - k/2)^s$. This function is a convex function in X whenever $s \geq 0$ is an even integer and $0 \leq X \leq 2k$, and hypergeometric random variables are so-called *Poisson binomial* random variables (they are sums of independent coin flip results with possibly different probabilities of “success”, where 1 marks a “success” and 0 marks a “failure”) (Vatutin and Mikhailov 1982)⁹², so by Hoeffding (1963)⁹³, when s is even, this central moment is no more than the s -th central moment of Y , a binomial random variable expressing the number of “successful” samples among k independent samples with “success” probability $1/2$; Y has the same mean as X , namely $k/2$.

In turn, the first inequality of Theorem 1 of Adell and Cárdenas-Morales (2018)⁹⁴ gives an upper bound of binomial central moments for even s when the “success” probability is $1/2$.

By that result, if s is an even integer then—

$$\tau_s(X) \leq \frac{s!}{8^{s/2}((s/2)!)} k^{s/2} \leq \frac{s!}{8^{s/2}((s/2)!)} n^{s/2},$$

where the last inequality uses that $k \leq n$.

The result for odd integers follows from a straightforward application of Cauchy’s inequality.

I acknowledge the user “Barbab” on *Cross Validated* for giving me useful insights. []

The following claim is believed to be true.

Claim. Suppose r is 3, suppose f is a function satisfying the conjecture’s hypothesis, suppose W_n is a sequence meeting that hypothesis, and suppose further that, for each n , W_n —

⁹⁰S. Spektor, “**Restricted Khinchine Inequality**”, *Canadian Mathematical Bulletin* 59(1), 2016, proposition 3.1, proved the bound $\tau_s(X) \leq \sqrt{2}(s/4)^{s/2} n^{s/2}$ assuming $s \geq 2$ is a real number and $k \geq n$ is an integer (and the result extends to $0 \leq k < n$ by symmetry), but this bound is looser than the one given by this lemma. <https://doi.org/10.4153/CMB-2015-047-8>

⁹¹Johnson, N.L., Kemp, A.W., Kotz, S., *Univariate Discrete Distributions, Third Edition*, Wiley, 2005.

⁹²Vatutin, V. A., and V. G. Mikhailov. “Limit theorems for the number of empty cells in an equiprobable scheme for group allocation of particles.” *Theory of Probability & Its Applications* 27.4 (1983): 734-743. <https://doi.org/10.1137/1127084>

⁹³Hoeffding, W., “Probability inequalities for sums of bounded random variables”, *Journal of the American Statistical Association* 58 (1963). <https://doi.org/10.1080/01621459.1963.10500830>

⁹⁴Adell, J.A., Cárdenas-Morales, D., “**Quantitative generalized Voronovskaja’s formulae for Bernstein polynomials**”, *Journal of Approximation Theory* 231, July 2018. <https://www.sciencedirect.com/science/article/pii/S0021904518300376>

1. is a linear operator,
2. satisfies $\text{abs}(f(\lambda) - W_n(\lambda)) \leq DM/n^{r/2}$ for some $D > 0$,
3. preserves all polynomials of degree $(r-1)$ or less (so that $W_n(x) = f(x)$ whenever f is a polynomial of degree $(r-1)$ or less and n is greater than or equal to f 's degree), and
4. has a continuous r -th derivative that satisfies $\text{abs}(W_n^{(r)}(\lambda)) \leq EM$ for some $E > 0$.

Then—

$$h(n, k) \leq \frac{\sqrt{2}DM(1+2\sqrt{2})}{4n^{\frac{3}{2}}} + \frac{\sqrt{3}EM}{48n^{\frac{3}{2}}}.$$

Remark: Because of this claim's assumptions, if $W_n(f)$ is continuous, W_n must be a *nonpositive operator* (so that, for some i , $W_i(f)$ can be somewhere negative even if f is nowhere negative) (Gavrea and Ivan 2018⁹⁵, Guessab et al. 2009⁹⁶); moreover, $W_n = f$ doesn't meet the conjecture's hypothesis (for $r \geq 3$).

Provisional proof: Because W_n is a linear operator, the left-hand side of (PB) is not greater than—

$$\begin{aligned} & \text{abs}\left(\left(\sum_{i=0}^k \left(W_n\left(\frac{i}{n}\right)\right)\sigma_{n,k,i}\right) - W_n(k/(2n))\right) + \\ & \text{abs}(W_n(k/(2n)) - f(k/(2n))) + \text{abs}(W_{2n}(k/(2n)) - f(k/(2n))). \end{aligned}$$

Because W_n also satisfies point 2, the left-hand side of (PB) is not greater than—

$$\text{abs}\left(\left(\sum_{i=0}^k \left(W_n\left(\frac{i}{n}\right)\right)\sigma_{n,k,i}\right) - W_n(k/(2n))\right) + \frac{DM(2^{r/2} + 1)}{2^{r/2}} \frac{1}{n^{r/2}}.$$

Let X be a hypergeometric($2n, k, n$) random variable. W_n has a continuous r -th derivative, and has a Lipschitz-continuous $(r-1)$ -th derivative, and the expected value or “long-run average” of X/n is $k/(2n)$, so by Lemmas 2B and 2C—

$$\begin{aligned} & \left(\sum_{i=0}^k \left(W_n\left(\frac{i}{n}\right)\right)\sigma_{n,k,i}\right) \\ & \leq \frac{\max(\text{abs}(W_n^{(2)}))\text{Var}[X/n]}{2} + \frac{EM \cdot \tau_3(X/n)}{6}. \end{aligned}$$

Because W_n is a linear operator that preserves polynomials of degree 2 or less, it can be assumed without loss of generality that W_n and the first and second derivatives of f equal zero at $k/(2n)$ (so that W_n 's *Taylor polynomial* centered at $k/(2n)$ is zero). Therefore—

$$\left(\sum_{i=0}^k \left(W_n\left(\frac{i}{n}\right)\right)\sigma_{n,k,i}\right) \leq \frac{EM \cdot \tau_3(X/n)}{6},$$

It remains to estimate the third central absolute moment of X/n .

⁹⁵Ioan Gavrea, Mircea Ivan, “A note on the fixed points of positive linear operators”, Journal of Approximation Theory (227), 2018, <https://doi.org/10.1016/j.jat.2017.12.001>.

⁹⁶Guessab, A., Nouisser, O. & Schmeisser, G. Enhancement of the algebraic precision of a linear operator and consequences under positivity. Positivity 13, 693–707 (2009). <https://doi.org/10.1007/s11117-008-2253-4>

$\tau_s(X/n) = \tau_s(X)/n^s$ for every $s \geq 0$, and Corollary C2 gives the result that $\tau_3(X) \leq (\sqrt{3}/8)n^{3/2}$, so $\tau_3(X/n) = \tau_3(X)/n^3 \leq (\sqrt{3}/8)/n^{3/2}$ and—

$$\text{abs}((\sum_{i=0}^k (W_n(\frac{i}{n}))\sigma_{n,k,i}) - W_n(k/(2n))) \leq \frac{EM}{6} \frac{\sqrt{3}}{8n^{3/2}},$$

so the left-hand side of (PB) is not greater than—

$$\frac{EM\sqrt{3}}{48n^{3/2}} + \frac{DM(2^{3/2} + 1)}{2^{3/2}} \frac{1}{n^{3/2}} \leq \frac{\sqrt{2}DM(1 + 2\sqrt{2})}{4n^{\frac{3}{2}}} + \frac{\sqrt{3}EM}{48n^{\frac{3}{2}}}.$$

9.4.2 Example of Polynomial-Building Scheme

The following example uses the results earlier in this appendix to build a polynomial-building scheme for a factory function.

Let $f(\lambda) = 0$ if λ is 0, and $1/(\ln(\lambda/\exp(3)))^2$ otherwise. (This function is not Hölder continuous; its slope is exponentially steep at the point 0.) Then the following scheme is valid in the sense of Theorem 1:

- $\eta(k) = \Phi(1, 2, (\ln(k) + \ln(7) + 6)/\ln(2))^2$.
- **fbelow**(n, k) = $f(k/n)$
- **fabove**(n, k) = $\max(\mathbf{fabove}(4,0), \mathbf{fabove}(4,1), \dots, \mathbf{fabove}(4,4))$ if $n < 4$; otherwise, $f(k/n) + \eta(n)$.

Where $\Phi(\cdot)$ is a function called the *Lerch transcendent*.

The first step is to find a concave modulus of continuity of f (called $\omega(h)$). Because f is strictly increasing and concave, and because $f(0) = 0$, $\omega(h) = f(h)$ can be taken for ω .

Now, plugging $\text{sqrt}(1/(7*n))$ into ω leads to the following for Theorem 2 (assuming $n \geq 0$):

- $\phi(n) = 1/(\ln(\text{sqrt}(7/n)/7) - 3)^2$.

Now, by applying Theorem 1, compute $\eta(k)$ by substituting n with 2^n , summing over all integers starting with k , and substituting k with $\ln(k)/\ln(2)$. η converges, resulting in:

- $\eta(k) = \Phi(1, 2, (\ln(k) + \ln(7) + 6)/\ln(2))^2$,

where $\Phi(\cdot)$ is the Lerch transcendent. This η matches the η given in the scheme above. That scheme then follows from Theorems 1 and 2, as well as from part 1 of Proposition 1 because f is concave.

The following code in Python that uses the SymPy computer algebra library is an example of finding the parameters for this polynomial-building scheme.

```
px=Piecewise((0,Eq(x,0)),((ln(x/exp(3))**-2),True))
<h1>omega is modulus of continuity. Since</h1>

<h1>px is strictly increasing, concave, and px(0)=0,</h1>

<h1>take omega as px</h1>

omega=px
omega=piecewise_fold(omega.rewrite(Piecewise)).simplify()
<h1>compute omega</h1>

phi=omega.subs(x,sqrt(1/(7*n)))
```

```

pprint(phi)
<h1>compute eta</h1>

eta=summation(phi.subs(n,2**n),(n,k,oo)).simplify()
eta=eta.subs(k,log(k,2)) # Replace k with ln(k)/ln(2)
pprint(eta)
for i in range(20):
    # Calculate upper bounds for eta at certain points.
    try:
        print("eta(2~%d) ~= %s" % (i,ceiling(eta.subs(k,2**i)*10000000).n()/10000000))
    except:
        print("eta(2~%d) ~= [FAILED]" % (i))

```

9.5 Which functions don't require outside randomness to simulate?

The function $f(\lambda)$ is *strongly simulable* if it admits a Bernoulli factory algorithm that uses nothing but the input coin as a source of randomness (Keane and O'Brien 1994)⁹⁷. See “**Randomized vs. Non-Randomized Algorithms**”⁹⁸.

In this section, f is a known function whose domain is the closed unit interval or a subset of that interval, and λ is a probability of heads that is known to lie in f 's domain but is otherwise unknown.

Strong Simulability Statement. A function $f(\lambda)$ is strongly simulable only if—

1. f is constant on its domain, or is continuous and polynomially bounded on its domain, and
2. f maps the closed unit interval or a subset of it to the closed unit interval, and
3. $f(0)$ equals 0 or 1 whenever 0 is in the domain of f , and
4. $f(1)$ equals 0 or 1 whenever 1 is in the domain of f .

Note: A function $f(\lambda)$ is *polynomially bounded* if both $f(\lambda)$ and $1 - f(\lambda)$ are greater than or equal to $\min(\lambda^n, (1 - \lambda)^n)$ for some integer n (Keane and O'Brien 1994)⁹⁹. For examples, see “**About Bernoulli Factories**”¹⁰⁰.

Keane and O'Brien already showed that f is strongly simulable if conditions 1 and 2 are true and neither 0 nor 1 are included in the domain of f . Conditions 3 and 4 are required because λ (the probability of heads) can be 0 or 1 so that the input coin returns 0 or 1, respectively, every time. This is called a “degenerate” coin. When given just a degenerate coin, no algorithm can produce one value with probability greater than 0, and another value with the opposite probability. Rather, the algorithm can only produce a constant value with probability 1. In the Bernoulli factory problem, that constant is either 0 or 1, so a Bernoulli factory algorithm for f must return 1 with probability 1, or 0 with probability 1, when given just a degenerate coin and no outside randomness, resulting in conditions 3 and 4.

To show that f is strongly simulable, it's enough to show that there is a Bernoulli factory for f that must flip the input coin and get 0 and 1 before it uses any outside randomness.

Proposition 1. *If $f(\lambda)$ is described in the strong simulability statement and is a polynomial with computable Bernstein coefficients, it is strongly simulable.*

⁹⁷Keane, M. S., and O'Brien, G. L., “A Bernoulli factory”, *ACM Transactions on Modeling and Computer Simulation* 4(2), 1994. <https://doi.org/10.1145/175007.175019>

⁹⁸https://peteroupc.github.io/bernoulli.html#Randomized_vs_Non_Randomized_Algorithms

⁹⁹Keane, M. S., and O'Brien, G. L., “A Bernoulli factory”, *ACM Transactions on Modeling and Computer Simulation* 4(2), 1994. <https://doi.org/10.1145/175007.175019>

¹⁰⁰https://peteroupc.github.io/bernoulli.html#About_Bernoulli_Factories

Proof: If f is the constant 0 or 1, the proof is trivial: simply return 0 or 1, respectively.

Otherwise: Let $a[j]$ be the j^{th} Bernstein coefficient of the polynomial in Bernstein form. Consider the following algorithm, modified from (Goyal and Sigman 2012)¹⁰¹.

1. Flip the input coin n times, and let j be the number of times the coin returned 1 this way.
2. If 0 is in the domain of f and if j is 0, return $f(0)$. (By condition 3, $f(0)$ must be either 0 or 1.)
3. If 1 is in the domain of f and if j is n , return $f(1)$. (By condition 4, $f(1)$ must be either 0 or 1.)
4. With probability $a[j]$, return 1. Otherwise, return 0. (For example, generate a uniformly distributed random variate, greater than 0 and less than 1, then return 1 if that variate is less than $a[j]$, or 0 otherwise. $a[j]$ is the Bernstein coefficient j of the polynomial written in Bernstein form), or 0 otherwise.

(By the properties of the Bernstein form, $a[0]$ will equal $f(0)$ and $a[n]$ will equal $f(1)$ whenever 0 or 1 is in the domain of f , respectively.)

Step 4 is done by first generating fair coin flips (such as with the von Neumann trick of flipping the input coin twice until the flip returns 0 then 1 or 1 then 0 this way, then taking the result as 0 or 1, respectively (von Neumann 1951)¹⁰²), then using the algorithm in “**Digit Expansions**”¹⁰³ to produce the probability $a[j]$. The algorithm computes $a[j]$ bit by bit and compares the computed value with the generated bits. Since the coin returned both 0 and 1 in step 1 earlier in the algorithm, we know the coin isn’t degenerate, so that step 4 will finish with probability 1. Now, since the Bernoulli factory used only the input coin for randomness, this shows that f is strongly simulable. []

Proposition 2. *If $f(\lambda)$ is described in the strong simulability statement, and if either f is constant on its domain or f meets the additional conditions below, then f is strongly simulable.*

1. *If $f(0) = 0$ or $f(1) = 0$ or both, then there is a polynomial $g(\lambda)$ whose Bernstein coefficients are computable and in the closed unit interval, such that $g(0) = f(0)$ and $g(1) = f(1)$ whenever 0 or 1, respectively, is in the domain of f , and such that $g(\lambda) > f(\lambda)$ for every λ in the domain of f , except at 0 and 1.*
2. *If $f(0) = 1$ or $f(1) = 1$ or both, then there is a polynomial $h(\lambda)$ whose Bernstein coefficients are computable and in the closed unit interval, such that $h(0) = f(0)$ and $h(1) = f(1)$ whenever 0 or 1, respectively, is in the domain of f , and such that $h(\lambda) < f(\lambda)$ for every λ in the domain of f , except at 0 and 1.*

Lemma 1. *If $f(\lambda)$ is described in the strong simulability statement and meets the additional condition below, then f is strongly simulable.*

- *There is a polynomial $g(\lambda)$ whose Bernstein coefficients are computable and in the closed unit interval, such that $g(0) = f(0)$ and $g(1) = f(1)$ whenever 0 or 1, respectively, is in the domain of f , and such that $g(\lambda) > f(\lambda)$ for every λ in the domain of f , except at 0 and 1.*

Proof: Consider the following algorithm.

1. If f is 0 everywhere in its domain or 1 everywhere in its domain, return 0 or 1, respectively.
2. Otherwise, use the algorithm given in Proposition 1 to simulate $g(\lambda)$. If the algorithm returns 0, return 0. By the additional condition in the lemma, 0 will be returned if λ is either 0 or 1.

Now, it is known that the input coin’s probability of heads is neither 0 nor 1.

By the conditions in the lemma, both $f(\lambda) > 0$ and $g(\lambda) > 0$ whenever $0 < \lambda < 1$ and λ is in f ’s domain.

¹⁰¹Vineet Goyal and Karl Sigman. 2012. On simulating a class of Bernstein polynomials. ACM Trans. Model. Comput. Simul. 22, 2, Article 12 (March 2012), 5 pages. <https://doi.org/10.1145/2133390.2133396>

¹⁰²von Neumann, J. “Various Techniques Used in Connection with Random Digits.” NBS Appl. Math. Ser. 12, pp. 36-38, 1951.

¹⁰³https://peteroupc.github.io/bernoulli.html#Digit_Expansion

Now let $h(\lambda) = f(\lambda)/g(\lambda)$. By the conditions in the lemma, h will be positive everywhere in that interval.

3. Return 1 if h has the following property: $h(\lambda) = 0$ whenever $0 < \lambda < 1$ and λ is in f 's domain.
4. Otherwise, the procedure runs a Bernoulli factory algorithm for $h(\lambda)$ that uses the input coin (and possibly outside randomness). Since h is continuous and polynomially bounded and the input coin's probability of heads is neither 0 nor 1, h is strongly simulable; the von Neumann trick lets the input coin serve as a fair coin, thus doing away with outside randomness in the algorithm.

Thus, f admits an algorithm that uses nothing but the input coin as a source of randomness, and so is strongly simulable. []

Lemma 2. *If $f(\lambda)$ is described in the strong simulability statement and meets the additional conditions below, then f is strongly simulable.*

1. *There are two polynomials $g(\lambda)$ and $\omega(\lambda)$ such that both polynomials' Bernstein coefficients are computable and all in the closed unit interval.*
2. *$g(0) = \omega(0) = f(0) = 0$ (so that 0 is in the domain of f).*
3. *$g(1) = \omega(1) = f(1) = 1$ (so that 1 is in the domain of f).*
4. *For every λ in the domain of f , except at 0 and 1, $g(\lambda) > f(\lambda)$.*
5. *For every λ in the domain of f , except at 0 and 1, $\omega(\lambda) < f(\lambda)$.*

Proof: First, assume g and ω have the same degree. If not, rewrite the polynomial with lesser degree to a polynomial in Bernstein form with the same degree as the other polynomial.

Now, let $g[j]$ and $\omega[j]$ be the j^{th} Bernstein coefficient of the polynomial g or ω , respectively. Consider the following algorithm, which is similar to the algorithm in Proposition 1.

1. Flip the input coin n times, and let j be the number of times the coin returned 1 this way.
2. If 0 is in the domain of f and if j is 0, return $g(0) = \omega(0) = 0$.
3. If 1 is in the domain of f and if j is n , return $g(1) = \omega(1) = 1$.
4. Generate a uniformly distributed random variate, greater than 0 and less than 1, then return 1 if that variate is less than $\omega[j]$, or return 0 if that variate is greater than $g[j]$. This step is carried out via the von Neumann method, as in Proposition 1.

If the algorithm didn't return a value, then by now we know that the input coin's probability of heads is neither 0 nor 1, since step 2 returned a value (either 0 or 1), which can only happen if the input coin didn't return all zeros or all ones.

Now let $r(\lambda) = (f(\lambda) - \omega(\lambda)) / (g(\lambda) - \omega(\lambda))$. By the conditions in the lemma, $h(\lambda)$ will be positive wherever $0 < \lambda < 1$ and λ is in the domain of f .

Now, run a Bernoulli factory algorithm for $r(\lambda)$ that uses the input coin (and possibly outside randomness). Since r is continuous and polynomially bounded and the input coin's probability of heads is neither 0 nor 1, r is strongly simulable; the von Neumann trick lets the input coin serve as a fair coin, thus doing away with outside randomness in the algorithm.

Thus, f admits an algorithm that uses nothing but the input coin as a source of randomness, and so is strongly simulable. []

Proof of Proposition 2: The following cases can occur:

1. If neither 0 nor 1 are in the domain of f , then f is strongly simulable by the discussion above.
2. If f is 0 everywhere in its domain or 1 everywhere in its domain: Return 0 or 1, respectively.
3. If 0 but not 1 is in the domain of f : If $f(0) = 0$, apply Lemma 1. If $f(0) = 1$, apply Lemma 1, but take $f = 1 - f$ and return 1 minus the output of the lemma's algorithm (this will bring $f(0) = 0$ and satisfy the lemma.)

4. If 1 but not 0 is in the domain of f : If $f(1) = 0$, apply Lemma 1. If $f(1) = 1$, apply Lemma 1, but take $f = 1 - f$ and return 1 minus the output of the lemma's algorithm (this will bring $f(1) = 0$ and satisfy the lemma.)
5. $f(0) = f(1) = 0$: Apply Lemma 1.
6. $f(0) = f(1) = 1$: Apply Lemma 1, but take $f = 1 - f$ and return 1 minus the output of the lemma's algorithm.
7. $f(0) = 0$ and $f(1) = 1$: Apply Lemma 2.
8. $f(0) = 1$ and $f(1) = 0$: Apply Lemma 2, but take $f = 1 - f$ and return 1 minus the output of the lemma's algorithm.

[]

Proposition 3. *If $f(\lambda)$ is described in the strong simulability statement and is Lipschitz continuous, then f is strongly simulable.*

Lemma 3. *If $f(\lambda)$ is described in the strong simulability statement, is Lipschitz continuous, and is such that $f(0) = 0$ and $f(1) = 0$ whenever 0 or 1, respectively, is in the domain of f , then f is strongly simulable.*

Proof: If f is 0 everywhere in its domain or 1 everywhere in its domain: Return 0 or 1, respectively. Otherwise, let—

- M be the Lipschitz constant of f (its derivative's maximum of the absolute value if f is continuous), or a computable number greater than this.
- l be either 0 if 0 is in the domain of f , or 1 otherwise, and
- u be either 0 if 1 is in the domain of f , or 1 otherwise.

To build g , take its degree as $\text{ceil}(M)+1$ or greater (so that g 's Lipschitz constant is greater than M and g has $\text{ceil}(M) + 2$ Bernstein coefficients or greater), then set the first Bernstein coefficient as l , the last one as u , and the remaining ones as 1. (As a result, the polynomial g will have computable Bernstein coefficients.) Then g will meet the additional condition for Lemma 1 and the result follows from that lemma. []

Lemma 4. *If $f(\lambda)$ is described in the strong simulability statement, is Lipschitz continuous, and is such that $f(0) = 0$ and $f(1) = 1$ (so that 0 and 1 are in the domain of f), then f is strongly simulable.*

Proof: Let M and l be as in Lemma 3.

To build g and ω , take their degree as $\text{ceil}(M)+1$ or greater (so that their Lipschitz constant is greater than M and each polynomial has $\text{ceil}(M) + 2$ Bernstein coefficients), then for each polynomial, set its first Bernstein coefficient as l and its last as 1. The remaining Bernstein coefficients of g are set as 1 and the remaining ones of ω are set as 0. (As a result, the polynomial g will have computable Bernstein coefficients.) Then g and ω will meet the additional conditions for Lemma 2 and the result follows from that lemma. []

Proof of Proposition 3: In the proof of proposition 2, replace Lemma 1 and Lemma 2 with Lemma 3 and Lemma 4, respectively. []

Conjecture 1. *The conditions of Proposition 2 are necessary for $f(\lambda)$ to be strongly simulable.*

It's possible that the conditions given are not necessary, and stronger conditions hold; for example: “0 is not in the domain of f , or there is a number $\epsilon > 0$ and a Lipschitz-continuous function $g(x)$ on the interval $0 \leq x < \epsilon$ such that $g(x) = f(x)$ whenever x is in both g 's and f 's domains”. A tricky example of such a function is $f(x) = (\sin(1/x)/4 + 1/2) \cdot (1 - (1 - x)^n)$ for $n \geq 1$ (and $f(0) = 0$), which is strongly simulable at 0. ($(1 - x)^n$ is the probability of the biased coin showing zero n times in a row.)

9.6 Multiple-Output Bernoulli Factory

A related topic is a Bernoulli factory that takes a coin with unknown probability of heads λ and produces one or more independent samples of the probability $f(\lambda)$. This section calls it a *multiple-output Bernoulli*

factory.

Obviously, any single-output Bernoulli factory can produce multiple outputs by running itself multiple times. But for some functions f , it may be that producing multiple outputs at a time may use fewer input coin flips than producing one output multiple times.

Let a and b be real numbers satisfying $0 < a < b < 1$, such as $a=1/100$, $b=99/100$. Define the *entropy bound* as $h(f(\lambda))/h(\lambda)$ where $h(x) = -x \ln(x) - (1-x) \ln(1-x)$ is related to the Shannon entropy function. The question is:

*When the probability λ is such that $a \leq \lambda \leq b$, is there a multiple-output Bernoulli factory for $f(\lambda)$ with an expected (“long-run average”) number of input coin flips per sample that is arbitrarily close to the entropy bound? Call such a Bernoulli factory an **optimal factory**.*

(See Nacu and Peres (2005, Question 2)¹⁰⁴.)

So far, the following functions do admit an *optimal factory*:

- The functions λ and $1 - \lambda$.
- Constants c satisfying $0 \leq c \leq 1$. As Nacu and Peres (2005)¹⁰⁵ already showed, any such constant admits an optimal factory: turn flips of the input coin into *fair coin flips* (numbers that each equal 1 or 0 with equal probability) using Peres’s iterated von Neumann extractor (Peres 1992)¹⁰⁶, then build a binary tree that generates 1 with probability c and 0 otherwise (Knuth and Yao 1976)¹⁰⁷.

It is easy to see that if an *optimal factory* exists for $f(\lambda)$, then one also exists for $1 - f(\lambda)$: simply change all ones returned by the $f(\lambda)$ factory into zeros and vice versa.

Also, as Yuval Peres (Jun. 24, 2021) told me, there is an efficient multiple-output Bernoulli factory for $f(\lambda) = \lambda/2$: the key is to flip the input coin enough times to produce fair coin flips using his extractor (Peres 1992)¹⁰⁸, then multiply each fair coin flip with another input coin flip to get a sample from $\lambda/2$. Given that the sample is equal to 0, there are three possibilities that can “be extracted to produce more fair bits”: either the fair coin flip is 0, or the coin flip is 0, or both are 0.

This algorithm, though, doesn’t count as an *optimal factory*, and Peres described this algorithm only incompletely. By simulation and trial and error I found an improved version of the algorithm. It uses two randomness extractors (extractor 1 and extractor 2) that produce fair coin flips from bits that are not necessarily from fair coin flips (which is done using a method given later in this section). The extractors must be asymptotically optimal (they must approach the entropy limit as closely as desired); one example is the iterated von Neumann construction in Peres (1992)¹⁰⁹. The algorithm consists of doing the following in a loop until the desired number of outputs is generated.

1. If the number of outputs generated so far is divisible by 20, do the following:
 - Generate a fair coin using the method given later in this section. If that bit is zero, output 0, then repeat this step unless the desired number of outputs has been generated. If the bit is 1, flip the input coin and output the result.
2. Otherwise, do the following:

¹⁰⁴Nacu, Șerban, and Yuval Peres. “Fast simulation of new coins from old”, The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

¹⁰⁵Nacu, Șerban, and Yuval Peres. “Fast simulation of new coins from old”, The Annals of Applied Probability 15, no. 1A (2005): 93-115. <https://projecteuclid.org/euclid.aoap/1106922322>

¹⁰⁶Peres, Y., “Iterating von Neumann’s procedure for extracting random bits”, Annals of Statistics 1992,20,1, p. 590-597. <https://doi.org/10.1214/aos/1176348543> <https://projecteuclid.org/euclid.aos/1176348543>

¹⁰⁷Knuth, Donald E. and Andrew Chi-Chih Yao. “The complexity of nonuniform random number generation”, in *Algorithms and Complexity: New Directions and Recent Results*, 1976.

¹⁰⁸Mendo, Luis. “An asymptotically optimal Bernoulli factory for certain functions that can be expressed as power series.” *Stochastic Processes and their Applications* 129, no. 11 (2019): 4366-4384. <https://doi.org/10.1016/j.spa.2018.11.017> <https://www.sciencedirect.com/science/article/pii/S0304414918306768>

¹⁰⁹Peres, Y., “Iterating von Neumann’s procedure for extracting random bits”, Annals of Statistics 1992,20,1, p. 590-597. <https://doi.org/10.1214/aos/1176348543> <https://projecteuclid.org/euclid.aos/1176348543>

1. Generate a fair coin using the method given later in this section, call it fc . Then flip the input coin and call the result bc .
2. Output $fc * bc$.
3. (The following steps pass “unused” randomness to the extractor in a specific way to ensure correctness.) If fc is 0, and bc is 1, append 0 to extractor 2’s input bits.
4. If fc and bc are both 0, append 1 then 1 to extractor 2’s input bits.
5. If fc is 1 and bc is 0, append 1 then 0 to extractor 2’s input bits.

Inspired by Peres’s result with $\lambda/2$, the following algorithm is proposed. It works for every function writable as $D(\lambda)/E(\lambda)$, where—

- D is the polynomial $D(\lambda) = \sum_{i=0}^k \lambda^i (1 - \lambda)^{k-i} d[i]$,
- E is the polynomial $E(\lambda) = \sum_{i=0}^k \lambda^i (1 - \lambda)^{k-i} e[i]$,
- every $d[i]$ is less than or equal to the corresponding $e[i]$, and
- each $d[i]$ and each $e[i]$ is a nonnegative integer.

The algorithm is a modified version of the “block simulation” in Mossel and Peres (2005, Proposition 2.5)¹¹⁰, which also “extracts” residual randomness with the help of six asymptotically optimal randomness extractors. In the algorithm, let r be an integer such that, for every integer i in $[0, k]$, $e[i] < \text{choose}(k, i) * \text{choose}(2^*r, r)$.

1. Set $iter$ to 0.
2. Flip the input coin k times. Then build a bitstring $B1$ consisting of the coin flip results in the order they occurred. Let i be the number of ones in $B1$.
3. Generate 2^*r a fair coin using the method given later in this section. (Rather than flipping the input coin 2^*r times, as in the algorithm of Proposition 2.5.) Then build a bitstring $B2$ consisting of the coin flip results in the order they occurred.
4. If the number of ones in $B2$ is other than r : Translate $B1 + B2$ to an integer under mapping 1, then pass that number to extractor 2 ($\dagger$), then add 1 to $iter$, then go to step 2.
5. Translate $B1 + B2$ to an integer under mapping 2, call the integer β . If $\beta < d[i]$, pass β to extractor 3, then pass $iter$ to extractor 6, then give out a 1. Otherwise, if $\beta < e[i]$, pass $\beta - d[i]$ to extractor 4, then pass $iter$ to extractor 6, then give out a 0. Otherwise, pass $\beta - e[i]$ to extractor 5, then add 1 to $iter$, then go to step 2.

The mappings used in this algorithm are as follows:

1. A one-to-one mapping between—
 - bitstrings of length $k + 2^*r$ with fewer or greater than r ones among the last 2^*r bits, and
 - the integers in $[0, 2^k * (2^2 - \text{choose}(2^*r, r))]$.
2. A one-to-one mapping between—
 - bitstrings of length $k + 2^*r$ with exactly i ones among the first k bits and exactly r ones among the remaining bits, and
 - the integers in $[0, \text{choose}(k, i) * \text{choose}(2^*r, r)]$.

In this algorithm, a fair coin flip is generated as follows. Let m be an even integer that is 32 or greater (in general, the greater m is, the more efficient the overall algorithm is in terms of coin flips).

1. Use extractor 1 to extract outputs from $\text{floor}(n/m) * m$ inputs, where n is the number of input bits available to that extractor. Do the same for the remaining extractors.
2. If extractor 2 has at least one unused output bit, take an output and stop. Otherwise, repeat this step for the remaining extractors.
3. Flip the input coin at least m times, append the coin results to extractor 1’s inputs, and go to step 1.

Now consider the last paragraph of Proposition 2.5. If the input coin were flipped in step 2, the probability of—

¹¹⁰Mossel, Elchanan, and Yuval Peres. New coins from old: computing with unknown bias. *Combinatorica*, 25(6), pp.707-724, 2005. With an appendix by Christopher Hillar. <https://doi.org/10.1007/s00493-005-0043-1>

- giving out 1 in the algorithm's last step would be $P1 = \lambda^{r*}(1 - \lambda)^{r*}D(\lambda)$, and
- giving out either 0 or 1 in that step would be $P01 = \lambda^{r*}(1 - \lambda)^{r*}E(\lambda)$,

so that the algorithm would simulate $f(\lambda) = P1 / P01$. Observe that the $\lambda^{r*}(1 - \lambda)^{r*}$ cancels out in the division. Thus, we could replace flips of the input coin with fair coin flips and still simulate $f(\lambda)$; the $\lambda^{r*}(1 - \lambda)^{r*}$ above would then be $(1/2)^{2r}$.

While this algorithm is coin-flip-efficient, it is not believed to be an optimal factory, at least not without more work. In particular, a bigger savings of input coin flips could occur if $f(\lambda)$ maps each value a or greater and b or less to a small range of values, so that the algorithm could, for example, generate a uniform random variate between 0 and 1 using fair coin flips and see whether that variate lies outside that range of values — and thus produce a sample from $f(\lambda)$ without flipping the input coin again.

(†) For example, by translating the number to input bits via Pae's entropy-preserving binarization (Pae 2020)¹¹¹. But correctness might depend on how this is done; after all, the number of coin flips per sample must equal or exceed the entropy bound for every λ .

9.7 Pushdown Automata and Algebraic Functions

This section has mathematical proofs showing which kinds of algebraic functions (functions that can be a solution of a nonzero polynomial equation) can be simulated with a pushdown automaton (a state machine with a stack).

The following summarizes what can be established about these algebraic functions:

- $\sqrt{\lambda}$ can be simulated.
- Every rational function with rational Bernstein coefficients that maps the open interval $(0, 1)$ to itself can be simulated.
- If $f(\lambda)$ can be simulated, so can any Bernstein-form polynomial in the variable $f(\lambda)$ with Bernstein coefficients that can be simulated.
- If $f(\lambda)$ and $g(\lambda)$ can be simulated, so can $f(\lambda)*g(\lambda)$, $f(g(\lambda))$, and $g(f(\lambda))$.
- If a full-domain pushdown automaton (defined later) can generate words of a given length with a given probability (a *probability distribution* of word lengths), then the probability generating function for that distribution can be simulated, as well as for that distribution conditioned on a finite set or periodic infinite set of word lengths (for example, all odd word lengths only).
- If a stochastic context-free grammar (defined later) can generate a probability distribution of word lengths, and terminates with probability 1, then the probability generating function for that distribution can be simulated.
- Every quadratic irrational number between 0 and 1 can be simulated.

It is not yet known whether the following functions can be simulated:

- $\lambda^{1/p}$ for prime numbers p greater than 2. The answer may be no; Banderier and Drmota (2015)¹¹² proved results that show, among other things, that $\lambda^{1/p}$, where p is not a power of 2, is not a possible solution to $P(\lambda) = 0$, where $P(\lambda)$ is a polynomial whose “power” coefficients are nonnegative real numbers.
- $\min(\lambda, 1 - \lambda)$.

The following definitions are used in this section:

¹¹¹S. -I. Pae, “Binarization Trees and Random Number Generation,” in IEEE Transactions on Information Theory, vol. 66, no. 4, pp. 2581-2587, April 2020, <https://doi.org/10.1109/TIT.2019.2962480> Also: S. Pae, “Binarization Trees and Random Number Generation”, arXiv:1602.06058v2 [cs.DS], 2018. <https://arxiv.org/abs/1602.06058v2>

¹¹²Banderier, C. And Drmota, M., 2015. “Formulae and asymptotics for coefficients of algebraic functions.” Combinatorics, Probability and Computing, 24(1), pp.1-53.

1. A *pushdown automaton* has a finite set of *states* and a finite set of *stack symbols*, one of which is called *EMPTY*, and takes a coin that shows heads with an unknown probability. It starts at a given state and its stack starts with *EMPTY*. On each iteration:
 - The automaton flips the coin.
 - Based on the coin flip (HEADS or TAILS), the current state, and the top stack symbol, it moves to a new state (or keeps it unchanged) and replaces the top stack symbol with zero, one or two symbols. Thus, there are three kinds of *transition rules*:
 - $(state, flip, symbol) \rightarrow (state2, \{symbol2\})$: move to *state2*, replace top stack symbol with same or different one.
 - $(state, flip, symbol) \rightarrow (state2, \{symbol2, new\})$: move to *state2*, replace top stack symbol with *symbol2*, then *push* a new symbol (*new*) onto the stack.
 - $(state, flip, symbol) \rightarrow (state2, \{\})$: move to *state2*, *pop* the top symbol from the stack.

When the stack is empty, the machine stops, and returns either 0 or 1 depending on the state it ends up at. (Because each left-hand side has no more than one possible transition, the automaton is *deterministic*.)

2. A *full-domain pushdown automaton* means a pushdown automaton that terminates with probability 1 given a coin with probability of heads λ , for every λ greater than 0 and less than 1.
3. **PDA** is the class of functions $f(\lambda)$ that satisfy $0 < f(\lambda) < 1$ whenever $0 < \lambda < 1$, and can be simulated by a full-domain pushdown automaton. **PDA** also includes the constant functions 0 and 1.
4. **ALGRAT** is the class of functions that satisfy $0 < f(\lambda) < 1$ whenever $0 < \lambda < 1$, are continuous, and are algebraic over the rational numbers (they satisfy a nonzero polynomial system whose “power” coefficients are rational numbers; specifically, there is a nonzero polynomial $P(x, y)$ in two variables and whose “power” coefficients are rational numbers, such that $P(x, f(x)) = 0$ for every x in the domain of f). **ALGRAT** also includes the constant functions 0 and 1.
5. A *probability generating function* has the form $p_0 * \lambda^0 + p_1 * \lambda^1 + \dots$, where p_i is the probability of getting i .

Notes:

1. Mossel and Peres (2005)¹¹³ defined pushdown automata to start with a nonempty stack of *arbitrary* size, and to allow each rule to replace the top symbol with an *arbitrary* number of symbols. Both cases can be reduced to the definition in this section.
2. Pushdown automata, as defined here, are very similar to so-called *probabilistic right-linear indexed grammars* (Icard 2020)¹¹⁴ and can be translated to those grammars as well as to *probabilistic pushdown systems* (Etessami and Yannakakis 2009)¹¹⁵, as long as those grammars and systems use only transition probabilities that are rational numbers.

Proposition 0 (Mossel and Peres 2005¹¹⁶, Theorem 1.2): *A full-domain pushdown automaton can simulate a function that maps $(0, 1)$ to itself only if the function is in class **ALGRAT**.*

It is not known whether **ALGRAT** and **PDA** are equal, but the following can be established about **PDA**:

Lemma 1A: *Let $g(\lambda)$ be a function in the class **PDA**, and suppose a pushdown automaton F has two rules of the form $(state, HEADS, stacksymbol) \rightarrow RHS1$ and $(state, TAILS, stacksymbol) \rightarrow RHS2$, where*

¹¹³Mossel, Elchanan, and Yuval Peres. New coins from old: computing with unknown bias. *Combinatorica*, 25(6), pp.707-724, 2005. With an appendix by Christopher Hillar. <https://doi.org/10.1007/s00493-005-0043-1>

¹¹⁴Icard, Thomas F., “Calibrating generative models: The probabilistic Chomsky–Schützenberger hierarchy”, *Journal of Mathematical Psychology* 95 (2020): 102308. <https://doi.org/10.1016/j.jmp.2019.102308>

¹¹⁵Icard, Thomas F., “Calibrating generative models: The probabilistic Chomsky–Schützenberger hierarchy”, *Journal of Mathematical Psychology* 95 (2020): 102308. <https://doi.org/10.1016/j.jmp.2019.102308>

¹¹⁶Mossel, Elchanan, and Yuval Peres. New coins from old: computing with unknown bias. *Combinatorica*, 25(6), pp.707-724, 2005. With an appendix by Christopher Hillar. <https://doi.org/10.1007/s00493-005-0043-1>

state and *stacksymbol* are a specific state/symbol pair among the left-hand sides of F 's rules. Then there is a pushdown automaton that transitions to RHS1 with probability $g(\lambda)$ and to RHS2 with probability $1 - g(\lambda)$ instead.

Proof: If RHS1 and RHS2 are the same, then the conclusion holds and nothing has to be done. Thus assume RHS1 and RHS2 are different.

Let G be the full-domain pushdown automaton for g . Assume that machines F and G stop when they pop EMPTY from the stack. If this is not the case, transform both machines by renaming the symbol EMPTY to EMPTY ' ', adding a new symbol EMPTY ' ' and new starting state X0, and adding rules $(X0, \text{flip}, \text{EMPTY}) \rightarrow (\text{start}, \{\text{EMPTY ' '}\})$ and rule $(\text{state}, \text{flip}, \text{EMPTY}) \rightarrow (\text{state}, \{\})$ for all states other than X0, where *start* is the starting state of F or G , as the case may be.

Now, rename each state of G as necessary so that the sets of states of F and of G are disjoint (mutually exclusive). Then, add to F a new stack symbol EMPTY ' (or a name not found in the stack symbols of G , as the case may be). Then, for the following two pairs of rules in F , namely—

$(\text{state}, \text{HEADS}, \text{stacksymbol}) \rightarrow (\text{state2heads}, \text{stackheads})$, and $(\text{state}, \text{TAILS}, \text{stacksymbol}) \rightarrow (\text{state2tails}, \text{stacktails})$,

add two new states state_0 and state_1 that correspond to *state* and have names different from all other states, and replace that rule with the following rules:

$(\text{state}, \text{HEADS}, \text{stacksymbol}) \rightarrow (g\text{start}, \{\text{stacksymbol}, \text{EMPTY '}\})$, $(\text{state}, \text{TAILS}, \text{stacksymbol}) \rightarrow (g\text{start}, \{\text{stacksymbol}, \text{EMPTY '}\})$, $(\text{state}_0, \text{HEADS}, \text{stacksymbol}) \rightarrow (\text{state2heads}, \text{stackheads})$, $(\text{state}_0, \text{TAILS}, \text{stacksymbol}) \rightarrow (\text{state2heads}, \text{stackheads})$, $(\text{state}_1, \text{HEADS}, \text{stacksymbol}) \rightarrow (\text{state2tails}, \text{stacktails})$, and $(\text{state}_1, \text{TAILS}, \text{stacksymbol}) \rightarrow (\text{state2tails}, \text{stacktails})$,

where $g\text{start}$ is the starting state for G , and copy the rules of the automaton for G onto F , but with the following modifications:

- Replace the symbol EMPTY in G with EMPTY ' .
- Replace each state in G with a name distinct from all other states in F .
- Replace each rule in G of the form $(\text{state}, \text{flip}, \text{EMPTY '}) \rightarrow (\text{state2}, \{\})$, where *state2* is a final state of G associated with output 1, with the rule $(\text{state}, \text{flip}, \text{EMPTY '}) \rightarrow (\text{state}_1, \{\})$.
- Replace each rule in G of the form $(\text{state}, \text{flip}, \text{EMPTY '}) \rightarrow (\text{state2}, \{\})$, where *state2* is a final state of G associated with output 0, with the rule $(\text{state}, \text{flip}, \text{EMPTY '}) \rightarrow (\text{state}_0, \{\})$.

Then, the final states of the new machine are the same as those for the original machine F . []

Lemma 1B: *There are pushdown automata that simulate the probabilities 0 and 1.*

Proof: The probability 0 automaton has the rules $(\text{START}, \text{HEADS}, \text{EMPTY}) \rightarrow (\text{START}, \{\})$ and $(\text{START}, \text{TAILS}, \text{EMPTY}) \rightarrow (\text{START}, \{\})$, and its only state START is associated with output 0. The probability 1 automaton is the same, except START is associated with output 1. Both automata obviously terminate with probability 1. []

Because of Lemma 1A, it's possible to label each left-hand side of a pushdown automaton's rules with not just HEADS or TAILS, but also a rational number or another function in **PDA**, as long as for each state/symbol pair, the probabilities for that pair sum to 1. For example, rules like the following are now allowed:

$(\text{START}, 1/2, \text{EMPTY}) \rightarrow \dots$, $(\text{START}, \text{sqrt}(\lambda)/2, \text{EMPTY}) \rightarrow \dots$, $(\text{START}, (1 - \text{sqrt}(\lambda))/2, \text{EMPTY}) \rightarrow \dots$

Proposition 1A: *If $f(\lambda)$ is in the class **PDA**, then so is every polynomial in Bernstein form of degree n , where $n \geq 0$ is integer, with Bernstein coefficients $a[0], a[1], \dots, a[n]$ that are functions in the class **PDA**.*

Proof Sketch: This corresponds to a two-stage pushdown automaton that follows the algorithm of Goyal

and Sigman (2012)¹¹⁷: The first stage counts the number of “heads” shown when flipping the $f(\lambda)$ coin, and the second stage flips another coin that has success probability $a[i]$, where i is the number of “heads”. The automaton’s transitions take advantage of Lemma 1A. []

Proposition 1: *If $f(\lambda)$ and $g(\lambda)$ are functions in the class **PDA**, then so is their product, namely $f(\lambda)*g(\lambda)$.*

Proof: Special case of Proposition 1A with $n=1$, $f(\lambda)=f(\lambda)$, $a[0]=0$ (using Lemma 1B), and $a[1]=g(\lambda)$. []

Corollary 1A: *If $f(\lambda)$, $g(\lambda)$, and $h(\lambda)$ are functions in the class **PDA**, then so is $f(\lambda)*g(\lambda) + (1 - f(\lambda))*h(\lambda)$.*

Proof: Special case of Proposition 1A with $n=1$, $f(\lambda)=f(\lambda)$, $a[0]=h(\lambda)$, and $a[1]=g(\lambda)$. []

Proposition 2: *If $f(\lambda)$ and $g(\lambda)$ are functions in the class **PDA**, then so is their composition, namely $f(g(\lambda))$ or $(f \circ g)(\lambda)$.*

Proof: Let F be the full-domain pushdown automaton for f . For each state/symbol pair among the left-hand sides of F ’s rules, apply Lemma 1A to the automaton F , using the function g . Then the new machine F terminates with probability 1 because the original F and the original automaton for g do for every λ greater than 0 and less than 1, and because the automaton for g never outputs the same value with probability 0 or 1 for any λ greater than 0 or less than 1. Moreover, f is in class **PDA** by Theorem 1.2 of (Mossel and Peres 2005)¹¹⁸ because the machine is a full-domain pushdown automaton. []

Proposition 3: *Every rational function with rational Bernstein coefficients that maps the open interval $(0, 1)$ to itself is in class **PDA**.*

Proof: These functions can be simulated by a finite-state machine (Mossel and Peres 2005)¹¹⁹. This corresponds to a full-domain pushdown automaton that has no stack symbols other than EMPTY, never pushes symbols onto the stack, and pops the only symbol EMPTY from the stack whenever it transitions to a final state of the finite-state machine. []

Note: An unbounded stack size is necessary for a pushdown automaton to simulate functions that a finite-state machine can’t. With a bounded stack size, there is a finite-state machine where each state not only holds the pushdown automaton’s original state, but also encodes the contents of the stack (which is possible because the stack’s size is bounded); each operation that would push, pop, or change the top symbol transitions to a state with the appropriate encoding of the stack instead.

Proposition 4: *If a full-domain pushdown automaton can generate words with the same letter such that the length of each word follows a probability distribution, then that distribution’s probability generating function is in class **PDA**.*

Proof: Let F be a full-domain pushdown automaton. Add one state FAILURE, then augment F with a special “letter-generating” operation as follows. Add the following rule that pops all symbols from the stack:

$(\text{FAILURE}, \text{flip}, \text{stacksymbol}) \rightarrow (\text{FAILURE}, \{\}),$

and for each rule of the following form that transitions to a letter-generating operation (where S and T are arbitrary states):

$(S, \text{flip}, \text{stacksymbol}) \rightarrow (T, \text{newstack}),$

add another state S' (with a name that differs from all other states) and replace that rule with the following rules:

¹¹⁷Vineet Goyal and Karl Sigman. 2012. On simulating a class of Bernstein polynomials. ACM Trans. Model. Comput. Simul. 22, 2, Article 12 (March 2012), 5 pages. <https://doi.org/10.1145/2133390.2133396>

¹¹⁸Mossel, Elchanan, and Yuval Peres. New coins from old: computing with unknown bias. Combinatorica, 25(6), pp.707-724, 2005. With an appendix by Christopher Hillar. <https://doi.org/10.1007/s00493-005-0043-1>

¹¹⁹Mossel, Elchanan, and Yuval Peres. New coins from old: computing with unknown bias. Combinatorica, 25(6), pp.707-724, 2005. With an appendix by Christopher Hillar. <https://doi.org/10.1007/s00493-005-0043-1>

$(S, \text{flip}, \text{stacksymbol}) \rightarrow (S', \{\text{stacksymbol}\})$, $(S', \text{HEADS}, \text{stacksymbol}) \rightarrow (T, \text{newstack})$, and $(S', \text{TAILS}, \text{stacksymbol}) \rightarrow (\text{FAILURE}, \{\})$.

Then if the stack is empty upon reaching the FAILURE state, the result is 0, and if the stack is empty upon reaching any other state, the result is 1. By Dughmi et al. (2021)¹²⁰, the machine now simulates the distribution’s probability generating function. Moreover, the function is in class **PDA** by Theorem 1.2 of Mossel and Peres (2005)¹²¹ because the machine is a full-domain pushdown automaton. []

Define a *stochastic context-free grammar* as follows. The grammar consists of a finite set of *nonterminals* and a finite set of *letters*, and rewrites one nonterminal (the starting nonterminal) into a word. The grammar has three kinds of rules (in generalized Chomsky normal form (Etessami and Yannakakis 2009)¹²²):

- $X \rightarrow a$ (rewrite X to the letter a).
- $X \rightarrow_p (a, Y)$ (with rational probability p , rewrite X to the letter a followed by the nonterminal Y). For the same left-hand side, all the p values must sum to 1.
- $X \rightarrow (Y, Z)$ (rewrite X to the nonterminals Y and Z in that order).

Instead of a letter (such as a), a rule can use ε (the empty string). (The grammar is *context-free* because the left-hand side has only a single nonterminal, so that no context from the word is needed to parse it.)

Proposition 5: *Every stochastic context-free grammar can be transformed into a pushdown automaton. If the automaton is a full-domain pushdown automaton and the grammar has a one-letter alphabet, the automaton can generate words such that the length of each word follows the same distribution as the grammar, and that distribution’s probability generating function is in class **PDA**.*

Proof Sketch: In the equivalent pushdown automaton:

- $X \rightarrow a$ becomes the two rules— $(\text{START}, \text{HEADS}, X) \rightarrow (\text{letter}, \{\})$, and $(\text{START}, \text{TAILS}, X) \rightarrow (\text{letter}, \{\})$. Here, *letter* is either START or a unique state in F that “detours” to a letter-generating operation for a and sets the state back to START when finished (see Proposition 4). If a is ε , *letter* is START and no letter-generating operation is done.
- $X \rightarrow_{p_i} (a_i, Y_i)$ (all rules with the same nonterminal X) are rewritten to enough rules to transition to a letter-generating operation for a_i , and swap the top stack symbol with Y_i , with probability p_i , which is possible with just a finite-state machine (see Proposition 4) because all the probabilities are rational numbers (Mossel and Peres 2005)¹²³. If a_i is ε , no letter-generating operation is done.
- $X \rightarrow (Y, Z)$ becomes the two rules— $(\text{START}, \text{HEADS}, X) \rightarrow (\text{START}, \{Z, Y\})$, and $(\text{START}, \text{TAILS}, X) \rightarrow (\text{START}, \{Z, Y\})$.

Here, X is the stack symbol EMPTY if X is the grammar’s starting nonterminal. Now, assuming the automaton is full-domain, the rest of the result follows easily. For a single-letter alphabet, the grammar corresponds to a system of polynomial equations, one for each rule in the grammar, as follows:

- $X \rightarrow a$ becomes $X = 1$ if a is the empty string (ε), or $X = \lambda$ otherwise.
- For each nonterminal X , all n rules of the form $X \rightarrow_{p_i} (a_i, Y_i)$ become the equation $X = p_1 * \lambda_1 * Y_1 + p_2 * \lambda_2 * Y_2 + \dots + p_n * \lambda_n * Y_n$, where λ_i is either 1 if a_i is ε , or λ otherwise.
- $X \rightarrow (Y, Z)$ becomes $X = Y * Z$.

Solving this system for the grammar’s starting nonterminal, and applying Proposition 4, leads to the *probabil-*

¹²⁰Shaddin Dughmi, Jason Hartline, Robert D. Kleinberg, and Rad Niazadeh. 2021. Bernoulli Factories and Black-box Reductions in Mechanism Design. J. ACM 68, 2, Article 10 (April 2021), 30 pages. <https://doi.org/10.1145/3440988>

¹²¹Mossel, Elchanan, and Yuval Peres. New coins from old: computing with unknown bias. Combinatorica, 25(6), pp.707-724, 2005. With an appendix by Christopher Hillar. <https://doi.org/10.1007/s00493-005-0043-1>

¹²²Kousha Etessami and Mihalis Yannakakis. 2009. Recursive Markov chains, stochastic grammars, and monotone systems of nonlinear equations. J. ACM 56, 1, Article 1 (January 2009), 66 pages. <https://doi.org/10.1145/1462153.1462154>

¹²³Mossel, Elchanan, and Yuval Peres. New coins from old: computing with unknown bias. Combinatorica, 25(6), pp.707-724, 2005. With an appendix by Christopher Hillar. <https://doi.org/10.1007/s00493-005-0043-1>

ity generating function for the grammar's word distribution. (See also Flajolet et al. 2010¹²⁴, Icard 2020¹²⁵.)
[]

Example: The stochastic context-free grammar— $X \rightarrow 1/2 (a, X1), X1 \rightarrow (X, X2), X2 \rightarrow (X, X), X \rightarrow 1/2 (a, X3), X3 \rightarrow \varepsilon$, which encodes ternary trees (Flajolet et al. 2010)¹²⁶, corresponds to the equation $X = (1/2) * \lambda * X * X * X + (1/2) * \lambda * 1$, and solving this equation for X leads to the probability generating function for such trees, which is a complicated expression.

Notes:

1. A stochastic context-free grammar in which all the probabilities are $1/2$ is called a *binary stochastic grammar* (Flajolet et al. 2010)¹²⁷. If every probability is a multiple of $1/n$, then the grammar can be called an “ n -ary stochastic grammar”. It is even possible for a nonterminal to have two rules of probability λ and $(1 - \lambda)$, which are used when the input coin returns 1 (HEADS) or 0 (TAILS), respectively.
2. If a pushdown automaton simulates the function $f(\lambda)$, then f corresponds to a special system of equations, built as follows (Mossel and Peres 2005)¹²⁸; see also Esparza et al. (2004)¹²⁹. For each state of the automaton (call the state en), include the following equations in the system based on the automaton's transition rules:

- $(st, p, sy) \rightarrow (s2, \{\})$ becomes either $\alpha_{st,sy,en} = p$ if $s2$ is en , or $\alpha_{st,sy,en} = 0$ otherwise.
- $(st, p, sy) \rightarrow (s2, \{sy1\})$ becomes $\alpha_{st,sy,en} = p * \alpha_{s2,sy1,en}$.
- $(st, p, sy) \rightarrow (s2, \{sy1, sy2\})$ becomes $\alpha_{st,sy,en} = p * \alpha_{s2,sy2,_} \sigma[1] * \alpha_{\sigma[1],sy1,en} + \dots + p * \alpha_{s2,sy2,\sigma[n]} * \alpha_{\sigma[n],sy1,en}$, where $\sigma[i]$ is one of the machine's n states.

(Here, p is the probability of using the specified transition rule; the special value HEADS becomes λ , and the special value TAILS becomes $1 - \lambda$.) Now, each time multiple equations have the same left-hand side, combine them into one equation with the same left-hand side, but with the sum of their right-hand sides. Then, for every variable of the form $\alpha_{a,b,c}$ not yet present in the system, include the equation $\alpha_{a,b,c} = 0$. Then, for each final state fs that returns 1, solve the system for the variable $\alpha_{START,EMPTY,fs}$ (where START is the automaton's starting state) to get a solution (a function) that maps the open interval $(0, 1)$ to itself. (Each solve can produce multiple solutions, but only one of them will map that open interval to itself assuming every p is either HEADS or TAILS.) Finally, add all the solutions to get $f(\lambda)$.

3. Assume there is a pushdown automaton (F) that follows Definition 1 except it uses a set of N input letters (and not simply HEADS or TAILS), accepts an input word if the stack is empty, and rejects the word if the machine reaches a configuration without a transition rule. Then a pushdown automaton in the full sense of Definition 1 (G) can be built. In essence:
 1. Add a new FAILURE state, which when reached, pops all symbols from the stack.
 2. For each pair $(state, stacksymbol)$ for F , add a set of rules that generate one of the input letters (each letter i generated with probability $f_i(\lambda)$, which must be a function in **PDA**), then use the generated letter to perform the transition stated in the corresponding rule for F . If there is no such transition, transition to the FAILURE state instead.

¹²⁴Flajolet, P., Pelletier, M., Soria, M., “On Buffon machines and numbers”, arXiv:0906.5560 [math.PR], 2010 <https://arxiv.org/abs/0906.5560>

¹²⁵Levy, H., *Stochastic dominance*, 1998.

¹²⁶Flajolet, P., Pelletier, M., Soria, M., “On Buffon machines and numbers”, arXiv:0906.5560 [math.PR], 2010 <https://arxiv.org/abs/0906.5560>

¹²⁷Flajolet, P., Pelletier, M., Soria, M., “On Buffon machines and numbers”, arXiv:0906.5560 [math.PR], 2010 <https://arxiv.org/abs/0906.5560>

¹²⁸Mossel, Elchanan, and Yuval Peres. New coins from old: computing with unknown bias. *Combinatorica*, 25(6), pp.707-724, 2005. With an appendix by Christopher Hillar. <https://doi.org/10.1007/s00493-005-0043-1>

¹²⁹Esparza, J., Kučera, A. and Mayr, R., “Model checking probabilistic pushdown automata.” In *Proceedings of the 19th Annual IEEE Symposium on Logic in Computer Science*, 2004. (pp. 12-21). IEEE. <https://doi.org/10.1109/LICS.2004.1319596>

3. When the stack is empty, output 0 if G is in the FAILURE state, or 1 otherwise.

Then G returns 1 with the same probability as F accepts an input word with letters randomly generated as in the second step. Also, one of the N letters can be a so-called “end-of-string” symbol, so that a pushdown automaton can be built that accepts “empty strings”; an example is Elder et al. (2015)¹³⁰.

Proposition 6: *If a full-domain pushdown automaton can generate a distribution of words with the same letter, there is a full-domain pushdown automaton that can generate a distribution of such words conditioned on—*

1. *a finite set of word lengths, or*
2. *a periodic infinite set of word lengths.*

One example of a finite set of word lengths is $\{1, 3, 5, 6\}$, where only words of length 1, 3, 5, or 6 are allowed. A *periodic infinite set* is defined by a finite set of integers such as $\{1\}$, as well as an integer modulus such as 2, so that in this example, all integers congruent to 1 modulo 2 (that is, all odd integers) are allowed word lengths and belong to the set.

Proof Sketch:

1. As in Lemma 1A, assume that the automaton stops when it pops EMPTY from the stack. Let S be the finite set (for example, $\{1, 3, 5, 6\}$), and let M be the maximum value in the finite set. For each integer i in $[0, M]$, make a copy of the automaton and append the integer i to the name of each of its states. Combine the copies into a new automaton F , and let its start state be the start state for copy 0. Now, whenever F generates a letter, instead of transitioning to the next state after the letter-generating operation (see Proposition 4), transition to the corresponding state for the next copy (for example, if the operation would transition to copy 2’s version of “XYZ”, namely “2_XYZ”, transition to “3_XYZ” instead), or if the last copy is reached, transition to the last copy’s FAILURE state. If F would transition to a failure state corresponding to a copy not in S (for example, “0_FAILURE”, “2_FAILURE”, “3_FAILURE” in this example), first all symbols other than EMPTY are popped from the stack and then F transitions to its start state (this is a so-called “rejection” operation). Now, all the final states (except FAILURE states) for the copies corresponding to the values in S (for example, copies 1, 3, 5, 6 in the example) are treated as returning 1, and all other states are treated as returning 0.
2. Follow (1), except as follows: (A) M is equal to the integer modulus minus 1. (B) For the last copy of the automaton, instead of transitioning to the next state after the letter-generating operation (see Proposition 4), transition to the corresponding state for copy 0 of the automaton. []

Proposition 7: *Every constant function equal to a quadratic irrational number between 0 and 1 is in class PDA.*

A *continued fraction* is one way to write a real number. For purposes of the following proof, every real number greater than 0 and less than 1 has the following *continued fraction expansion*: $0 + 1 / (a[1] + 1 / (a[2] + 1 / (a[3] + \dots)))$, where each $a[i]$, a *partial denominator*, is an integer greater than 0. A *quadratic irrational number* is an irrational number that can be written as $(b + \sqrt{c})/d$, where b , c , and d are rational numbers.

Proof: By Lagrange’s continued fraction theorem, every quadratic irrational number has a continued fraction expansion that is eventually periodic; the expansion can be described using a finite number of partial denominators, the last “few” of which repeat forever. The following example describes a periodic continued fraction expansion: $[0; 1, 2, (5, 4, 3)]$, which is the same as $[0; 1, 2, 5, 4, 3, 5, 4, 3, 5, 4, 3, \dots]$. In this example, the partial denominators are the numbers after the semicolon; the size of the period $((5, 4, 3))$ is 3; and the size of the nonperiod $(1, 2)$ is 2.

¹³⁰Elder, Murray, Geoffrey Lee, and Andrew Rechnitzer. “Permutations generated by a depth 2 stack and an infinite stack in series are algebraic.” *Electronic Journal of Combinatorics* 22(1), 2015.

Given a periodic expansion, and with the aid of an algorithm for simulating **continued fractions**¹³¹, a recursive Markov chain for the expansion (Etessami and Yannakakis 2009)¹³² can be described as follows. The chain's components are all built on the following template. The template component has one entry E, one inner node N, one box, and two exits X0 and X1. The box has one *call port* as well as two *return ports* B0 and B1.

- From E: Go to N with probability x , or to the box's call port with probability $1 - x$.
- From N: Go to X1 with probability y , or to X0 with probability $1 - y$.
- From B0: Go to E with probability 1.
- From B1: Go to X0 with probability 1.

Let p be the period size, and let n be the nonperiod size. Now the recursive Markov chain to be built has $n+p$ components:

- For each i in $[1, n+1]$, there is a component labeled i . It is the same as the template component, except $x = a[i]/(1 + a[i])$, and $y = 1/a[i]$. The component's single box goes to the component labeled $i+1$, *except* that for component $n+p$, the component's single box goes to the component labeled $n+1$.

According to Etessami and Yannakakis (2009)¹³³, the recursive Markov chain can be translated to a pushdown automaton of the kind used in this section. Now all that's left is to argue that the recursive Markov chain terminates with probability 1. For every component in the chain, it goes from its entry to its box with probability $1/2$ or less (because each partial numerator must be 1 or greater). Thus, the component recurses with no greater probability than not, and there are otherwise no probability-1 loops in each component, so the overall chain terminates with probability 1. []

Lemma 1: *The square root function $\text{sqrt}(\lambda)$ is in class **PDA**.*

Proof: See Mossel and Peres (2005)¹³⁴. []

Corollary 1: *The function $f(\lambda) = \lambda^m / (2^n)$, where n and m are positive integers, is in class **PDA**.*

Proof: Start with the case $m=1$. If n is 1, write f as $\text{sqrt}(\lambda)$; if n is 2, write f as $(\text{sqrt } \text{sqrt})(\lambda)$; and for general n , write f as $(\text{sqrt } \text{sqrt } \dots \text{sqrt})(\lambda)$, with n instances of sqrt . Because this is a composition and sqrt can be simulated by a full-domain pushdown automaton, so can f .

For general m and n , write f as $((\text{sqrt } \text{sqrt } \dots \text{sqrt})(\lambda))^m$, with n instances of sqrt . This involves doing m multiplications of $\text{sqrt } \text{sqrt } \dots \text{sqrt}$, and because this is an integer power of a function that can be simulated by a full-domain pushdown automaton, so can f .

Moreover, f is in class **PDA** by Theorem 1.2 of (Mossel and Peres 2005)¹³⁵ because the machine is a full-domain pushdown automaton. []

9.7.1 Finite-State and Pushdown Generators

Another interesting class of machines (called *pushdown generators* here) are similar to pushdown automata (see above), with the following exceptions:

1. Each transition rule can also, optionally, give out a base- N digit in its right-hand side. An example is: $(\text{state}, \text{flip}, \text{sy}) \rightarrow (\text{digit}, \text{state2}, \{\text{sy2}\})$.
2. The machine must give out infinitely many digits if allowed to run forever.

¹³¹https://peteroupc.github.io/bernoulli.html#Continued_Fractions

¹³²Kousha Etessami and Mihalis Yannakakis. 2009. Recursive Markov chains, stochastic grammars, and monotone systems of nonlinear equations. J. ACM 56, 1, Article 1 (January 2009), 66 pages. <https://doi.org/10.1145/1462153.1462154>

¹³³Kousha Etessami and Mihalis Yannakakis. 2009. Recursive Markov chains, stochastic grammars, and monotone systems of nonlinear equations. J. ACM 56, 1, Article 1 (January 2009), 66 pages. <https://doi.org/10.1145/1462153.1462154>

¹³⁴Mossel, Elchanan, and Yuval Peres. New coins from old: computing with unknown bias. Combinatorica, 25(6), pp.707-724, 2005. With an appendix by Christopher Hillar. <https://doi.org/10.1007/s00493-005-0043-1>

¹³⁵Mossel, Elchanan, and Yuval Peres. New coins from old: computing with unknown bias. Combinatorica, 25(6), pp.707-724, 2005. With an appendix by Christopher Hillar. <https://doi.org/10.1007/s00493-005-0043-1>

3. Rules that would pop the last symbol from the stack are not allowed.

The “output” of the machine is now a real number X in the form of the base- N digit expansion $0.\text{dddddd}\dots$, where $\text{dddddd}\dots$ are the digits produced by the machine from left to right. In the rest of this section:

- $\text{CDF}(\mathbf{z})$ is the cumulative distribution function of X , or the probability that X is z or less.
- $\text{PDF}(\mathbf{z})$ is the probability density function of X , or the derivative of $\text{CDF}(\mathbf{z})$, or the relative probability of choosing a number “close” to z at random.

A *finite-state generator* (Knuth and Yao 1976)¹³⁶ is the special case where the probability of heads is $1/2$, each digit is either 0 or 1, rules can’t push stack symbols, and only one stack symbol is used. Then if $\text{PDF}(\mathbf{z})$ has infinitely many derivatives on the open interval $(0, 1)$, it must be a polynomial with rational Bernstein coefficients and satisfy $\text{PDF}(\mathbf{z}) > 0$ whenever $0 \leq \mathbf{z} \leq 1$ is irrational (Vatan 2001)¹³⁷, (Kindler and Romik 2004)¹³⁸, and it can be shown that the expected value (mean or “long-run average”) of X must be a rational number.¹³⁹

Proposition 8. *Suppose a finite-state generator can generate a probability distribution that takes on finitely many values. Then:*

1. *Each value occurs with a rational probability.*
2. *Each value is either rational or transcendental.*

A real number is *transcendental* if it can’t be a root of a nonzero polynomial with integer “power” coefficients. Thus, part 2 means, for example, that irrational, nontranscendental numbers such as $1/\sqrt{2}$ and the golden ratio minus 1 can’t be generated exactly.

Proving this proposition involves the following lemma, which shows that a finite-state generator is related to a machine with a one-way read-only input and a one-way write-only output:

Lemma 2. *A finite-state generator can fit the model of a one-way transducer-like k -machine (as defined in Adamczewski et al. (2020)¹⁴⁰ section 5.3), for some k equal to 2 or greater.*

Proof Sketch: There are two cases.

Case 1: If every transition rule of the generator gives out a digit, then k is the number of unique inputs among the generator’s transition rules (usually, there are two unique inputs, namely HEADS and TAILS), and the model of a finite-state generator is modified as follows:

1. A *configuration* of the finite-state generator consists of its current state together with either the last coin flip result or, if the coin wasn’t flipped yet, the empty string.
2. The *output function* takes a configuration described earlier and returns a digit. If the coin wasn’t flipped yet, the function returns an arbitrary digit (which is not used in proposition 4.6 of the Adamczewski paper).

Case 2: If at least one transition rule does not give out a digit, then the finite-state generator can be transformed to a machine where HEADS/TAILS is replaced with 50% probabilities, then transformed to an equivalent machine whose rules always output one or more digits, as claimed in Lemma 5.2 of Vatan

¹³⁶Knuth, Donald E. and Andrew Chi-Chih Yao. “The complexity of nonuniform random number generation”, in *Algorithms and Complexity: New Directions and Recent Results*, 1976.

¹³⁷Farrokh Vatan. 2001. Distribution functions of probabilistic automata. In Proceedings of the thirty-third annual ACM symposium on Theory of computing (STOC ’01). Association for Computing Machinery, New York, NY, USA, 684–693. <https://doi.org/10.1145/380752.380872>

¹³⁸Kindler, Guy and D. Romik, “On distributions computable by random walks on graphs,” *SIAM Journal on Discrete Mathematics* 17 (2004): 624–633. <https://doi.org/10.1137/S089548010343106X>

¹³⁹Vatan (2001) claims that a finite-state generator has a continuous CDF (unless it produces a single value with probability 1), but this is not necessarily true if the generator has a state that outputs 0 forever.

¹⁴⁰Adamczewski, B., Cassaigne, J. and Le Gonidec, M., 2020. “On the computational complexity of algebraic numbers: the Hartmanis–Stearns problem revisited.” *Transactions of the American Mathematical Society*, 373(5), pp.3085–3115. <https://doi.org/10.1090/tran/7915>

(2001)¹⁴¹. In case the resulting generator has rules that output more than one digit, additional states and rules can be added so that the generator's rules output only one digit as desired. Now at this point the generator's probabilities will be rational numbers. Now transform the generator from probabilities to inputs of size k , where k is the number of unique probabilities, by adding additional rules as desired. []

Proof of Proposition 8: Let n be an integer greater than 0. Take a finite-state generator that starts at state START and branches to one of n finite-state generators (subgenerators) with some probability, which must be rational because the overall generator is a finite-state machine (Icard 2020, Proposition 13)¹⁴². The branching process outputs no digit, and part 3 of the proposition follows from Corollary 9 of Icard (2020)¹⁴³. The n subgenerators are special; each of them generates the binary expansion of a single real number in the closed unit interval with probability 1.

To prove part 2 of the proposition, translate an arbitrary finite-state generator to a machine described in Lemma 2. Once that is done, all that must be shown is that there are two different nonempty sequences of coin flips that end up at the same configuration. This is easy using the pigeonhole principle, since the finite-state generator has a finite number of configurations. Thus, by propositions 5.11, 4.6, and AB of Adamczewski et al. (2020)¹⁴⁴, the generator can generate a real number's binary expansion only if that number is rational or transcendental (see also Cobham (1968)¹⁴⁵; Adamczewski and Bugeaud (2007)¹⁴⁶). []

Proposition 9. *If the distribution function generated by a finite-state generator is continuous and algebraic on the open interval $(0, 1)$, then that function is a piecewise polynomial function on that interval.*

The proof follows from combining Kindler and Romik (2004, Theorem 2)¹⁴⁷ and Knuth and Yao (1976)¹⁴⁸ with Richman (2012)¹⁴⁹, who proved that a continuous algebraic function on an open interval is piecewise analytic, so that each piece is analytic at every point except possibly at the endpoints.

10 License

Any copyright to this page is released to the Public Domain. In case this is not possible, this page is also licensed under **Creative Commons Zero**¹⁵⁰.

¹⁴¹Farrokh Vatan. 2001. Distribution functions of probabilistic automata. In Proceedings of the thirty-third annual ACM symposium on Theory of computing (STOC '01). Association for Computing Machinery, New York, NY, USA, 684–693. <https://doi.org/10.1145/380752.380872>

¹⁴²Icard, Thomas F., “Calibrating generative models: The probabilistic Chomsky–Schützenberger hierarchy”, *Journal of Mathematical Psychology* 95 (2020): 102308. <https://doi.org/10.1016/j.jmp.2019.102308>

¹⁴³Icard, Thomas F., “Calibrating generative models: The probabilistic Chomsky–Schützenberger hierarchy”, *Journal of Mathematical Psychology* 95 (2020): 102308. <https://doi.org/10.1016/j.jmp.2019.102308>

¹⁴⁴Adamczewski, B., Cassaigne, J. and Le Gonidec, M., 2020. “On the computational complexity of algebraic numbers: the Hartmanis–Stearns problem revisited.” *Transactions of the American Mathematical Society*, 373(5), pp.3085–3115. <https://doi.org/10.1090/tran/7915>

¹⁴⁵Cobham, A., “On the Hartmanis–Stearns problem for a class of tag machines”, in *IEEE Conference Record of 1968 Ninth Annual Symposium on Switching and Automata Theory* 1968.

¹⁴⁶Adamczewski, Boris, and Yann Bugeaud. “On the Complexity of Algebraic Numbers I. Expansions in Integer Bases.” *Annals of Mathematics* 165, no. 2 (2007): 547–65. <http://www.jstor.org/stable/20160035>.

¹⁴⁷Kindler, Guy and D. Romik, “On distributions computable by random walks on graphs,” *SIAM Journal on Discrete Mathematics* 17 (2004): 624–633. <https://doi.org/10.1137/S089548010343106X>

¹⁴⁸Knuth, Donald E. and Andrew Chi-Chih Yao. “The complexity of nonuniform random number generation”, in *Algorithms and Complexity: New Directions and Recent Results*, 1976.

¹⁴⁹Richman, F. (2012). Algebraic functions, calculus style. *Communications in Algebra*, 40(7), 2671–2683. <https://doi.org/10.1080/00927872.2011.584337>

¹⁵⁰<https://creativecommons.org/publicdomain/zero/1.0/>